Cloud Services and Security Assignment

Assignment - 8

Name: Tanmay Harish Chaudhari

Course: PGCP-HPCSA

PRN: 260840127026

Q.1: **Write a Terraform configuration to create an AWS EC2 instance using an Amazon Linux AMI.**

Output:

Main.tf

```
thc  …/css/tf/q1  08:14  ls
./  ../  main.tf  .terraform/  .terraform.lock.hcl  terraform.tfstate  terraform.tfstate.backup

thc  …/css/tf/q1  08:20  nvim main.tf

thc  …/css/tf/q1  08:20
```

nvim main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~>5.0"
    }
  }

}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "q1" {
  ami           = "ami-0f19324397f53458f"
  instance_type = "t2.nano"
  subnet_id     = "subnet-0904080bfac070e36"
  key_name      = "cdac-css-assignments"
  tags = {
    Name = "Terraform-EC2"
  }
}
```

Terraform Initialization

```
thc   …/css/tf/q1   08:02  nvim main.tf

thc   …/css/tf/q1   08:03  terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Terraform validate, format, plan and apply to aws

```
thc   …/css/tf/q1   08:12  terraform validate
Success! The configuration is valid.


thc   …/css/tf/q1   08:12  terraform fmt

thc   …/css/tf/q1   08:12  terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.q1 will be created
  + resource "aws_instance" "q1" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
```

```
      + public_dns                          = (known after apply)
      + public_ip                           = (known after apply)
      + secondary_private_ips               = (known after apply)
      + security_groups                     = (known after apply)
      + source_dest_check                   = true
      + spot_instance_request_id            = (known after apply)
      + subnet_id                           = "subnet-0904080bfac070e36"
      + tags                                = {
          + "Name" = "Terraform-EC2"
        }
      + tags_all                            = {
          + "Name" = "Terraform-EC2"
        }
      + tenancy                             = (known after apply)
      + user_data                           = (known after apply)
      + user_data_base64                    = (known after apply)
      + user_data_replace_on_change         = false
      + vpc_security_group_ids              = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

thc   …/css/tf/q1   08:12  terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.q1 will be created
  + resource "aws_instance" "q1" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
```

```
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "Terraform-EC2"
        }
      + tags_all                             = {
          + "Name" = "Terraform-EC2"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.q1: Creating...
```

```
aws_instance.q1: Creating...
aws_instance.q1: Still creating... [00m10s elapsed]
aws_instance.q1: Creation complete after 14s [id=i-0537c9696cc196a4a]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

thc  …/css/tf/q1  08:12  terraform destroy
aws_instance.q1: Refreshing state... [id=i-0537c9696cc196a4a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.q1 will be destroyed
  - resource "aws_instance" "q1" {
      - ami                                  = "ami-0f19324397f53458f" -> null
      - arn                                  = "arn:aws:ec2:ap-south-1:985539764528:instance/i-0537c9696cc196a4a" -> null
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "ap-south-1a" -> null
      - cpu_core_count                       = 1 -> null
      - cpu_threads_per_core                 = 1 -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - get_password_data                    = false -> null
      - hibernation                          = false -> null
      - id                                   = "i-0537c9696cc196a4a" -> null
      - instance_initiated_shutdown_behavior = "stop" -> null
      - instance_state                       = "running" -> null
      - instance_type                        = "t2.nano" -> null
      - ipv6_address_count                   = 0 -> null
      - ipv6_addresses                       = [] -> null
      - key_name                             = "cdac-css-assignments" -> null
      - monitoring                           = false -> null
      - placement_partition_number           = 0 -> null
      - primary_network_interface_id         = "eni-0c9dc250883295971" -> null
      - private_dns                          = "ip-10-0-0-179.ap-south-1.compute.internal" -> null
      - private_ip                           = "10.0.0.179" -> null
      - public_ip                            = "43.205.115.180" -> null
      - secondary_private_ips                = [] -> null
      - security_groups                      = [] -> null
      - source_dest_check                    = true -> null
      - subnet_id                            = "subnet-0904080bfac070e36" -> null
      - tags                                 = {
          - "Name" = "Terraform-EC2"
        } -> null
```

```
      - tags_all                              = {
          - "Name" = "Terraform-EC2"
        } -> null
      - tenancy                               = "default" -> null
      - user_data_replace_on_change           = false -> null
      - vpc_security_group_ids                = [
          - "sg-0f31f6de4774f5dec",
        ] -> null
        # (8 unchanged attributes hidden)

      - capacity_reservation_specification {
          - capacity_reservation_preference = "open" -> null
        }

      - cpu_options {
          - core_count       = 1 -> null
          - threads_per_core = 1 -> null
            # (1 unchanged attribute hidden)
        }

      - credit_specification {
          - cpu_credits = "standard" -> null
        }

      - enclave_options {
          - enabled = false -> null
        }

      - maintenance_options {
          - auto_recovery = "default" -> null
        }

      - metadata_options {
          - http_endpoint               = "enabled" -> null
          - http_protocol_ipv6          = "disabled" -> null
          - http_put_response_hop_limit = 2 -> null
          - http_tokens                 = "required" -> null
          - instance_metadata_tags      = "disabled" -> null
        }

      - private_dns_name_options {
          - enable_resource_name_dns_a_record    = true -> null
          - enable_resource_name_dns_aaaa_record = false -> null
          - hostname_type                        = "ip-name" -> null
        }

      - root_block_device {
```

```
      - root_block_device {
          - delete_on_termination = true -> null
          - device_name           = "/dev/xvda" -> null
          - encrypted             = false -> null
          - iops                  = 3000 -> null
          - tags                  = {} -> null
          - tags_all              = {} -> null
          - throughput            = 125 -> null
          - volume_id             = "vol-04d0adfa945757071" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp3" -> null
            # (1 unchanged attribute hidden)
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

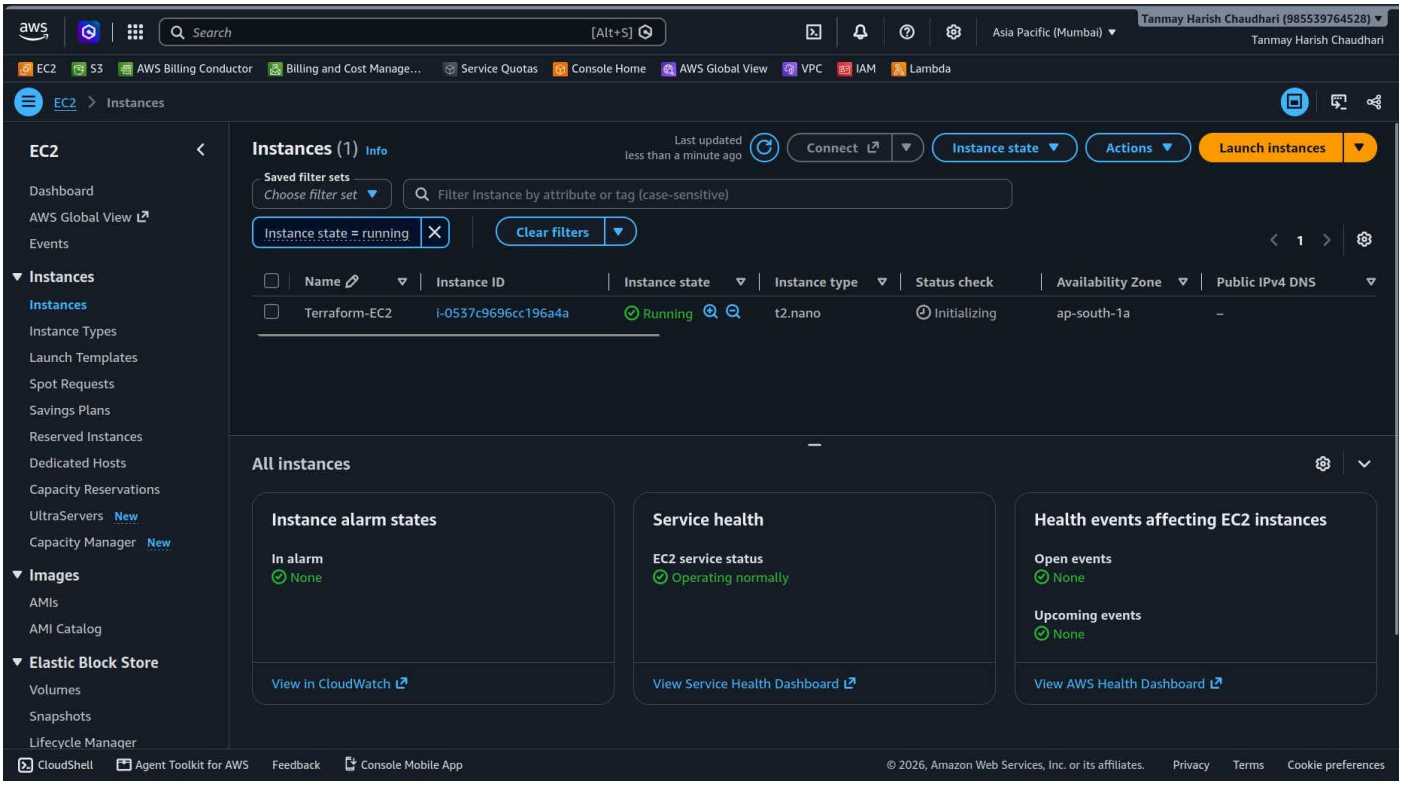
  Enter a value: yes

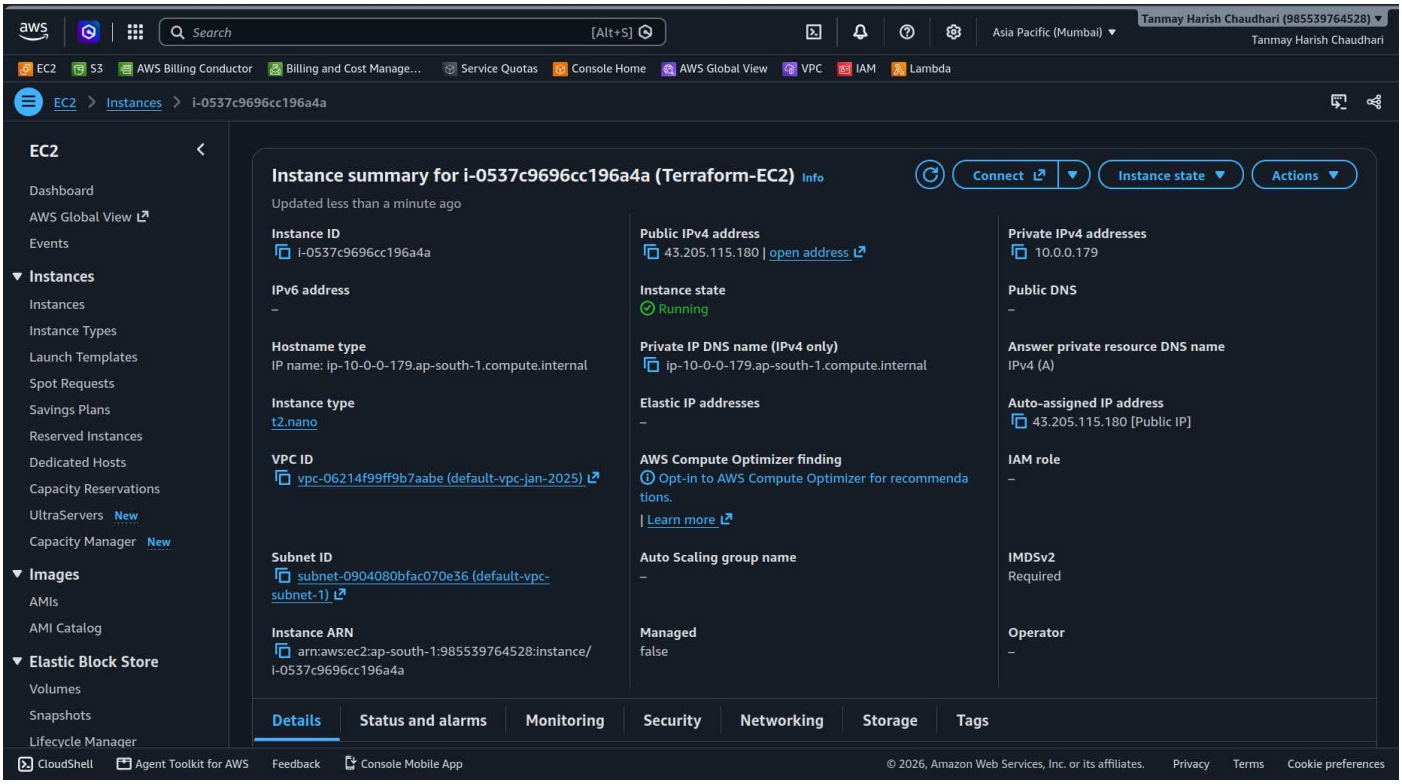
aws_instance.q1: Destroying... [id=i-0537c9696cc196a4a]
aws_instance.q1: Still destroying... [id=i-0537c9696cc196a4a, 00m10s elapsed]
aws_instance.q1: Still destroying... [id=i-0537c9696cc196a4a, 00m20s elapsed]
aws_instance.q1: Destruction complete after 21s

Destroy complete! Resources: 1 destroyed.

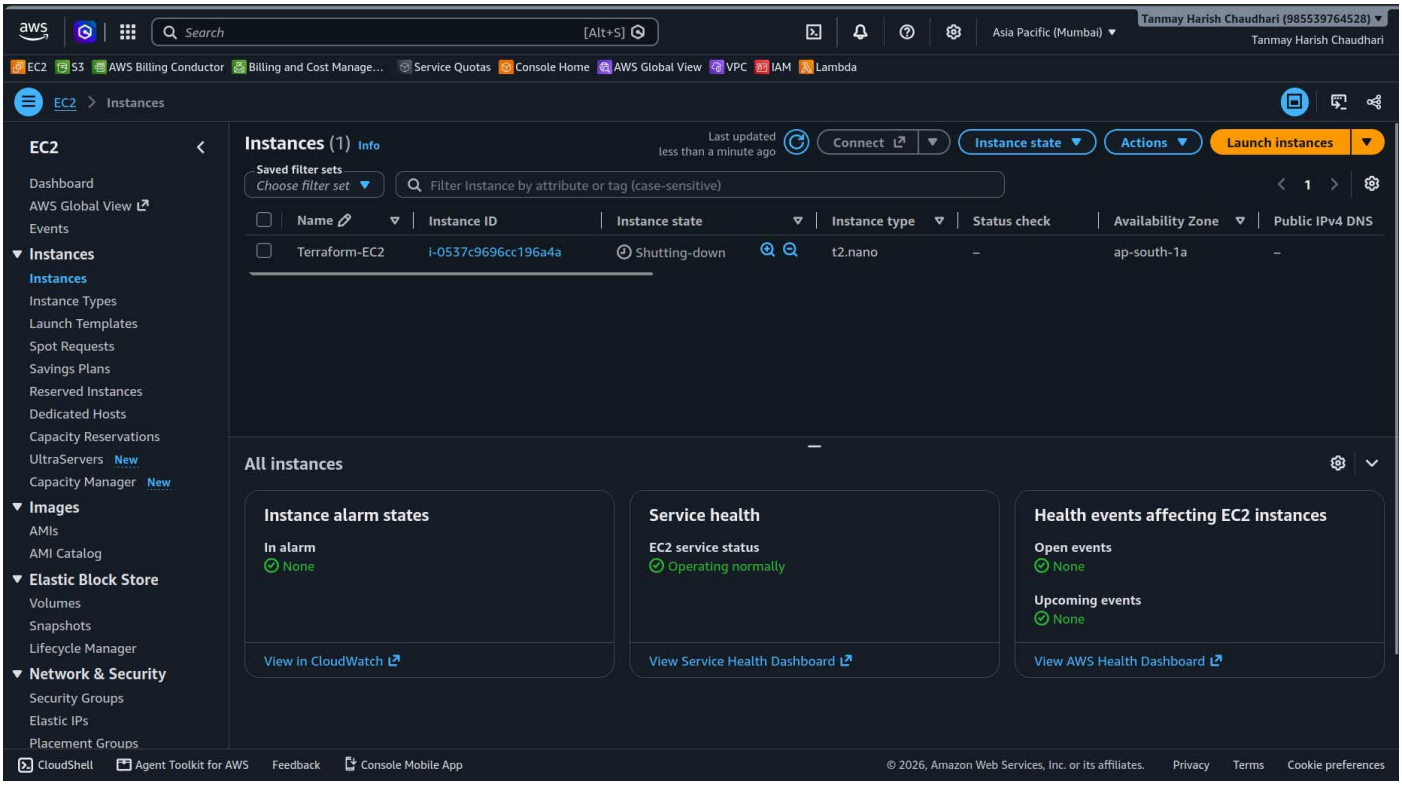
thc   …/css/tf/q1  08:14
```

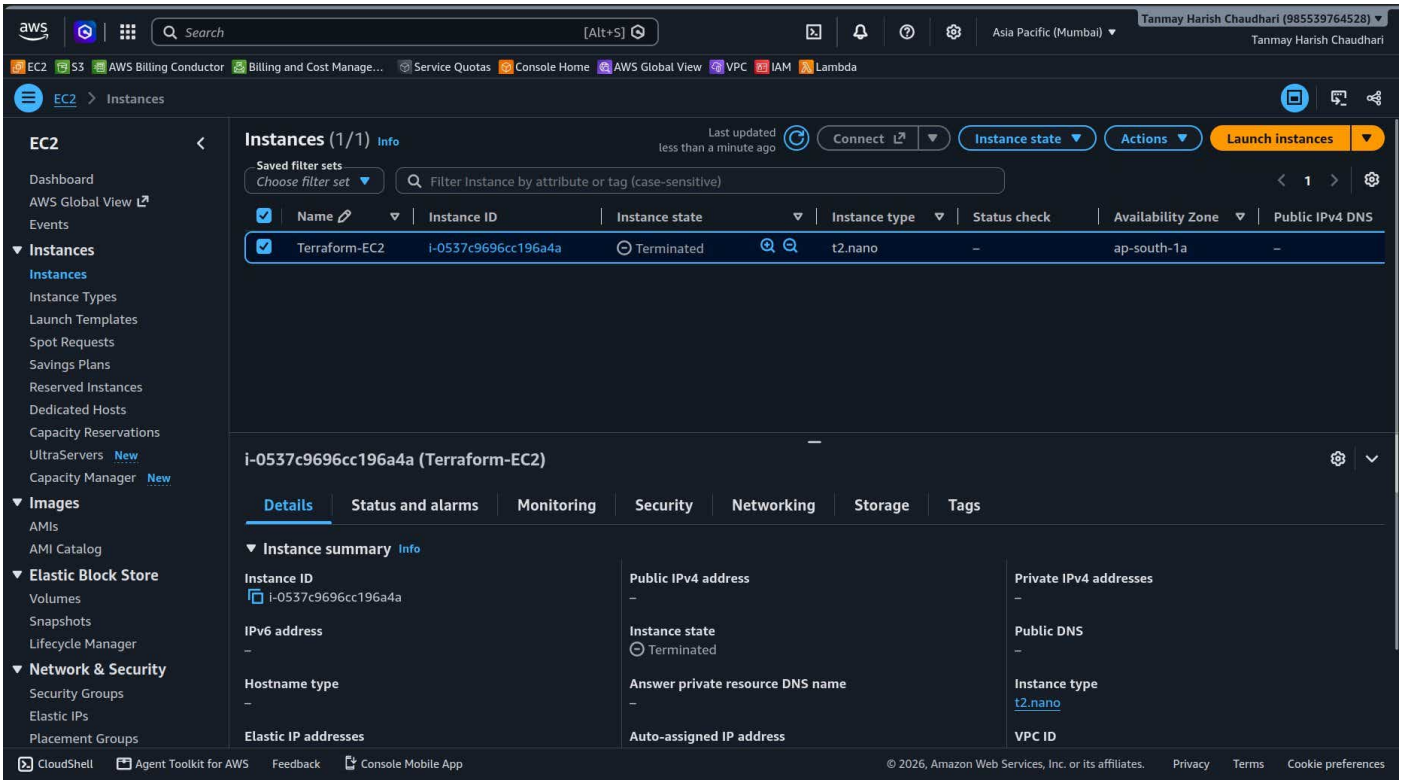
Terraform Apply





Terraform Destroy





Q.2: **Create an AWS S3 bucket using Terraform and upload a sample file to it.**

Output:

```
thc   ~/css/tf   08:22  mkdir q2

thc   ~/css/tf   08:22  cd q2
./  ../

thc   …/css/tf/q2   08:22  nvim main.tf

thc   …/css/tf/q2   08:28  nvim main.tf

thc   …/css/tf/q2   08:28  nvim Sample.txt

thc   …/css/tf/q2   08:29  cat Sample.txt
```

	File: Sample.txt
1	This is sample text file for css assignment"

Terraform initialize, format and validate and plan

```
thc   …/css/tf/q2   08:29  nvim main.tf

thc   …/css/tf/q2   08:29  terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc   …/css/tf/q2   08:29  terraform fmt
main.tf

thc   …/css/tf/q2   08:30  terraform validate
Success! The configuration is valid.


thc   …/css/tf/q2   08:30  terraform p
plan       providers  push

thc   …/css/tf/q2   08:30  terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.sample_bucket will be created
  + resource "aws_s3_bucket" "sample_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
```

```
      + arn                         = (known after apply)
      + bucket                      = "260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = (known after apply)
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

  # aws_s3_object.file_upload will be created
  + resource "aws_s3_object" "file_upload" {
      + acl                    = (known after apply)
      + arn                    = (known after apply)
      + bucket                 = "aws_s3_bucket.260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_key_enabled     = (known after apply)
      + checksum_crc32         = (known after apply)
      + checksum_crc32c        = (known after apply)
      + checksum_crc64nvme     = (known after apply)
      + checksum_sha1          = (known after apply)
      + checksum_sha256        = (known after apply)
      + content_type           = (known after apply)
```

Terraform Apply

```
      + content_type           = (known after apply)
      + etag                   = (known after apply)
      + force_destroy          = false
      + id                     = (known after apply)
      + key                    = "Sample.txt"
      + kms_key_id             = (known after apply)
      + server_side_encryption = (known after apply)
      + source                 = "Sample.txt"
      + storage_class          = (known after apply)
      + tags_all               = (known after apply)
      + version_id             = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

──────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform ap

thc   …/css/tf/q2   08:30   terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.sample_bucket will be created
  + resource "aws_s3_bucket" "sample_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = (known after apply)
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)
```

```
      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

  # aws_s3_object.file_upload will be created
  + resource "aws_s3_object" "file_upload" {
      + acl                    = (known after apply)
      + arn                    = (known after apply)
      + bucket                 = "aws_s3_bucket.260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_key_enabled     = (known after apply)
      + checksum_crc32         = (known after apply)
      + checksum_crc32c        = (known after apply)
      + checksum_crc64nvme     = (known after apply)
      + checksum_sha1          = (known after apply)
      + checksum_sha256        = (known after apply)
      + content_type           = (known after apply)
      + etag                   = (known after apply)
      + force_destroy          = false
      + id                     = (known after apply)
      + key                    = "Sample.txt"
      + kms_key_id             = (known after apply)
      + server_side_encryption = (known after apply)
      + source                 = "Sample.txt"
      + storage_class          = (known after apply)
      + tags_all               = (known after apply)
      + version_id             = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_object.file_upload: Creating...

╷
│ Error: uploading S3 Object (Sample.txt) to Bucket (sample_bucket.260840127026-tc-sample-file-bucket-css-assignment): operation error S3: PutObject, https response error St
atusCode: 404, RequestID: G8FVY7A63ZNW0M99, HostID: XhsLih/AirOIagPxy34NvVlHiisxQxDzT0m1M391x1B9pZedezrW79ZPA+7DOfVmW8lG+uEgsGo=, api error NoSuchBucket: The specified bucke
t does not exist
│
│   with aws_s3_object.file_upload,
│   on main.tf line 18, in resource "aws_s3_object" "file_upload":
│   18: resource "aws_s3_object" "file_upload" {
│
╵

thc  …/css/tf/q2  08:35  nvim main.tf
```

Reapplying due to improper bucket address for file_upload

```
thc   …/css/tf/q2   08:38  terraform apply
aws_s3_bucket.sample_bucket: Refreshing state... [id=260840127026-tc-sample-file-bucket-css-assignment]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_object.file_upload will be created
  + resource "aws_s3_object" "file_upload" {
      + acl                    = (known after apply)
      + arn                    = (known after apply)
      + bucket                 = "260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_key_enabled     = (known after apply)
      + checksum_crc32         = (known after apply)
      + checksum_crc32c        = (known after apply)
      + checksum_crc64nvme     = (known after apply)
      + checksum_sha1          = (known after apply)
      + checksum_sha256        = (known after apply)
      + content_type           = (known after apply)
      + etag                   = (known after apply)
      + force_destroy          = false
      + id                     = (known after apply)
      + key                    = "Sample.txt"
      + kms_key_id             = (known after apply)
      + server_side_encryption = (known after apply)
      + source                 = "Sample.txt"
      + storage_class          = (known after apply)
      + tags_all               = (known after apply)
      + version_id             = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

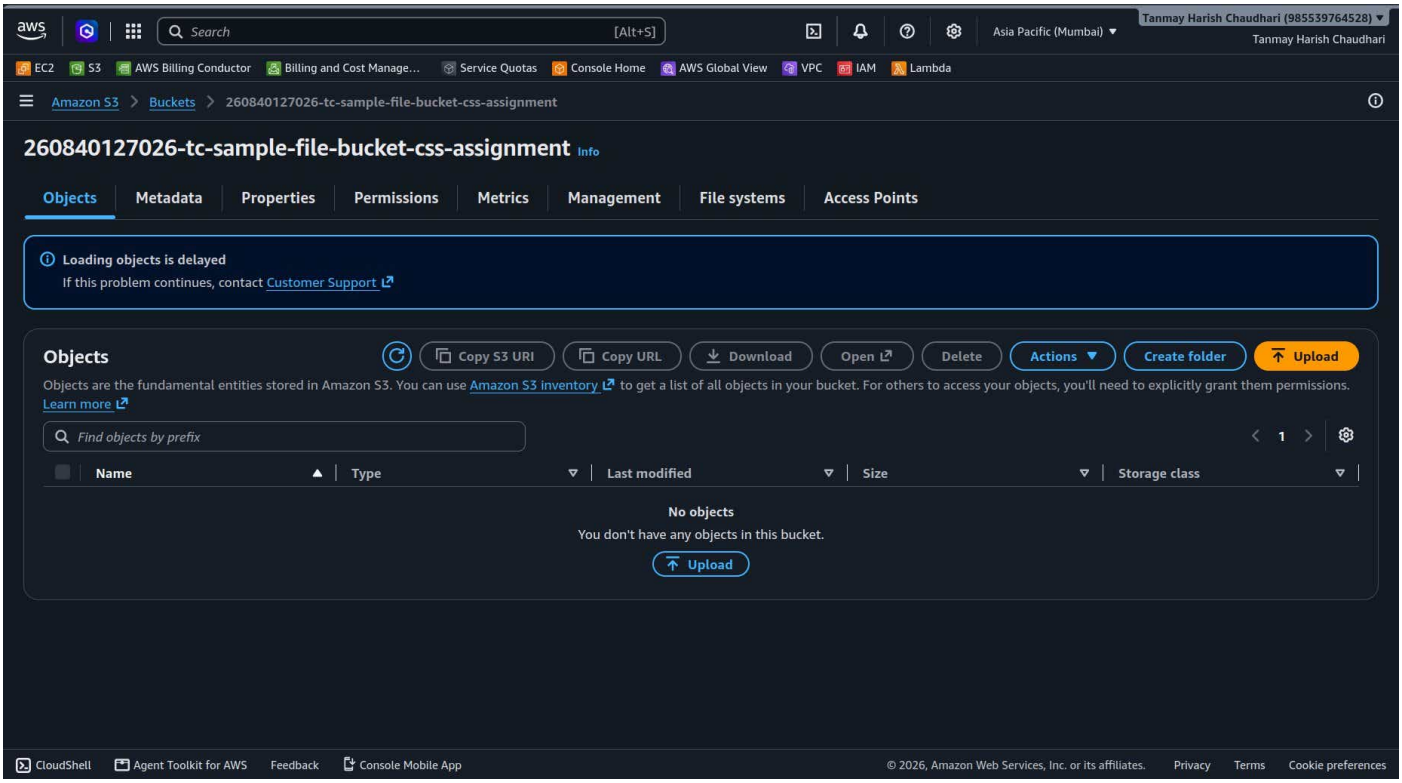
aws_s3_object.file_upload: Creating...
aws_s3_object.file_upload: Creation complete after 0s [id=Sample.txt]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

thc   …/css/tf/q2   08:38
```

nvim main.tf

```
18  terraform {
17    required_providers {
16      aws = {
15        source  = "hashicorp/aws"
14        version = "~>5.0"
13      }
12    }
11  }
10
 9  provider "aws" {
 8    region = "ap-south-1"
 7  }
 6
 5  resource "aws_s3_bucket" "sample_bucket" {
 4    bucket = "260840127026-tc-sample-file-bucket-css-assignment"
 3  }
 2
 1  resource "aws_s3_object" "file_upload" {
19    bucket = aws_s3_bucket.sample_bucket.id
 1    key    = "Sample.txt"
 2    source = "Sample.txt"
 3  }
```


Search [Alt+S]
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
Tanmay Harish Chaudhari
EC2 S3 AWS Billing Conductor Billing and Cost Manage... Service Quotas Console Home AWS Global View VPC IAM Lambda
Amazon S3 > Buckets > 260840127026-tc-sample-file-bucket-css-assignment
260840127026-tc-sample-file-bucket-css-assignment Info
Objects Metadata Properties Permissions Metrics Management File systems Access Points
Loading objects is delayed
If this problem continues, contact Customer Support
Objects
Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions.
Learn more
Find objects by prefix
1
Name Type Last modified Size Storage class
No objects
You don't have any objects in this bucket.
Upload
CloudShell Agent Toolkit for AWS Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Q.3: **Create an IAM user using Terraform and attach a policy that allows read-only access to S3.**

Output:

```
thc  …/css/tf/q3  08:46  nvim main.tf

thc  …/css/tf/q3  08:46  cat main.tf
```

File: main.tf

```
1   terraform{
2     required_providers{
3       aws = {
4         source = "hashicorp/aws"
5         version = "~>5.0"
6       }
7     }
8   }
9
10  provider "aws" {
11    region = "ap-south-1"
12  }
13
14  resource "aws_iam_user" "s3_reader" {
15    name = "s3-read-only-user"
16  }
17
18  resource "aws_iam_user_policy_attachment" "s3_read_only_attach" {
19    user       = aws_iam_user.s3_reader.name
20    policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
21  }
```

```
thc  …/css/tf/q3  08:46  terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc  …/css/tf/q3  08:46  terraform fmt
main.tf

thc  …/css/tf/q3  08:46  terraform
apply         env           get          init         metadata     plan         query        show         taint        validate
console       fmt           graph        login        modules      providers    refresh      stacks       test         version
destroy       force-unlock  import       logout       output       push         rpcapi       state        untaint      workspace

thc  …/css/tf/q3  08:46  terraform validate
Success! The configuration is valid.


thc  …/css/tf/q3  08:47  terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.s3_reader will be created
  + resource "aws_iam_user" "s3_reader" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "s3-read-only-user"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }

  # aws_iam_user_policy_attachment.s3_read_only_attach will be created
  + resource "aws_iam_user_policy_attachment" "s3_read_only_attach" {
      + id         = (known after apply)
      + policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
      + user       = "s3-read-only-user"
    }

Plan: 2 to add, 0 to change, 0 to destroy.
```

```
Plan: 2 to add, 0 to change, 0 to destroy.


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

thc   …/css/tf/q3   08:47  terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.s3_reader will be created
  + resource "aws_iam_user" "s3_reader" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "s3-read-only-user"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }

  # aws_iam_user_policy_attachment.s3_read_only_attach will be created
  + resource "aws_iam_user_policy_attachment" "s3_read_only_attach" {
      + id         = (known after apply)
      + policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
      + user       = "s3-read-only-user"
    }

Plan: 2 to add, 0 to change, 0 to destroy.

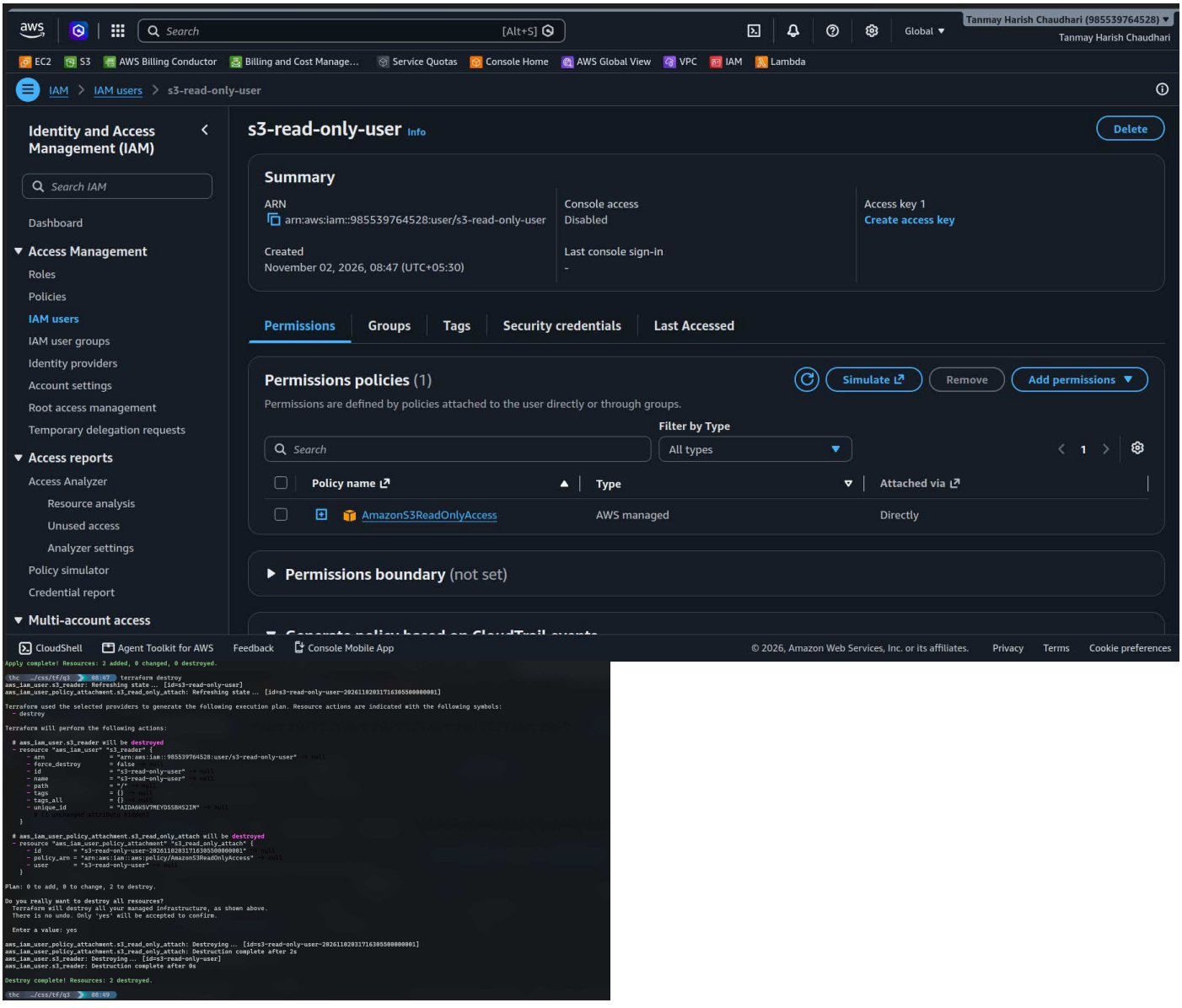
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_user.s3_reader: Creating...
aws_iam_user.s3_reader: Creation complete after 2s [id=s3-read-only-user]
aws_iam_user_policy_attachment.s3_read_only_attach: Creating...
aws_iam_user_policy_attachment.s3_read_only_attach: Creation complete after 1s [id=s3-read-only-user-20261102031716305500000001]

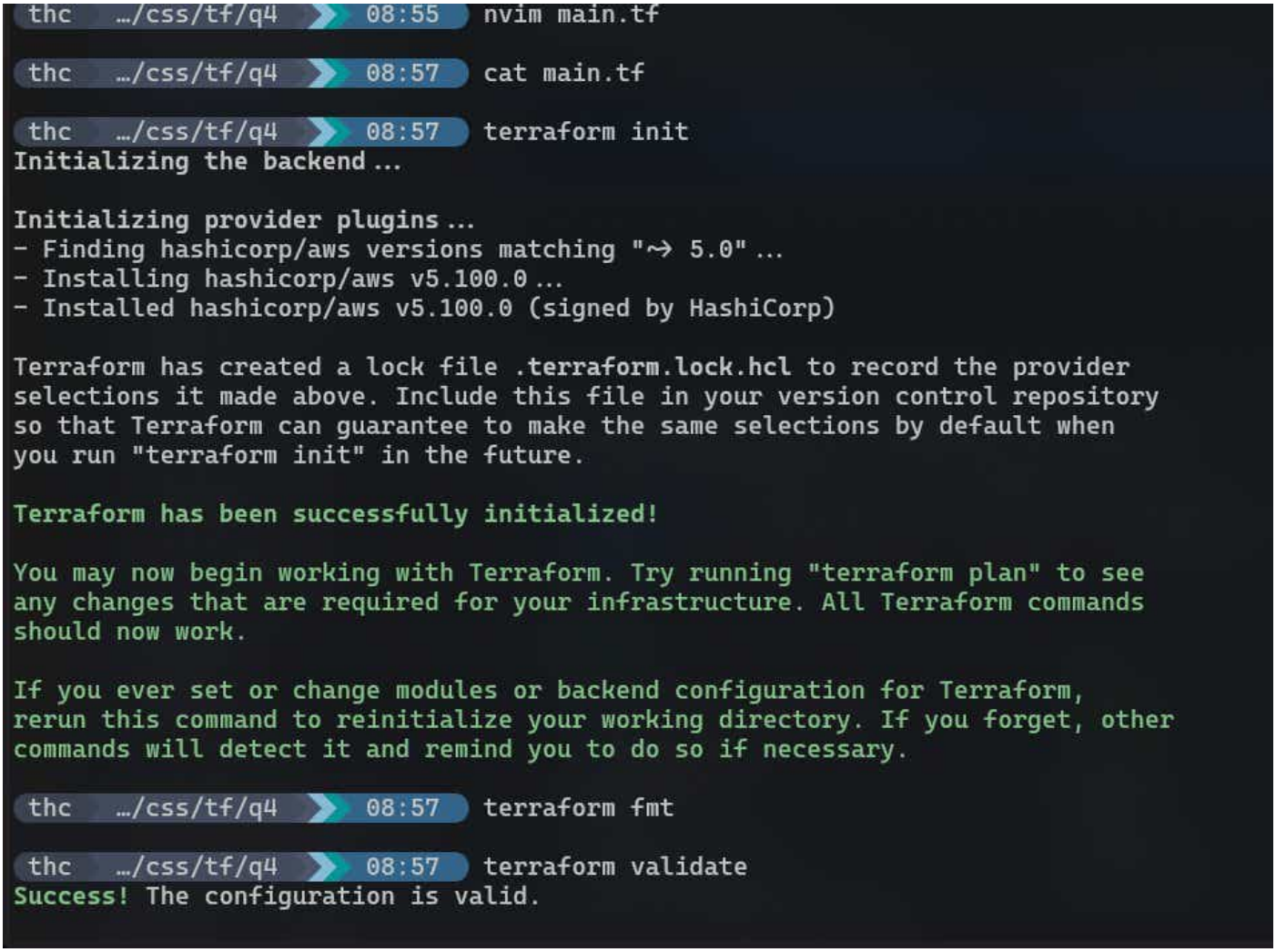
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

thc   …/css/tf/q3   08:47
```



Q.4: **Create an EC2 instance with a security group that allows SSH access on port 22 and HTTP access on port 80.**

Output:



main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_security_group" "web_ssh_sg" {
  name        = "web-ssh-security-group"
  vpc_id      = "vpc-06214f99ff9b7aabe"

  ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "web-ssh-sg"
  }
}

resource "aws_instance" "q1" {
  ami                         = "ami-0f19324397f53458f"
  instance_type               = "t2.nano"
  subnet_id                   = "subnet-0904080bfac070e36"
  key_name                    = "cdac-css-assignments"
  vpc_security_group_ids      = [aws_security_group.web_ssh_sg.id]
  associate_public_ip_address = true

  tags = {
    Name = "Terraform-EC2"
  }
}

output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.q1.public_ip
}
```

```
thc   …/css/tf/q4   08:58   nvim main.tf

thc   …/css/tf/q4   09:04   terraform fmt

thc   …/css/tf/q4   09:04   terraform validate
Success! The configuration is valid.


thc   …/css/tf/q4   09:04   terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.q1 will be created
  + resource "aws_instance" "q1" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "Terraform-EC2"
        }
      + tags_all                             = {
          + "Name" = "Terraform-EC2"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

  # aws_security_group.web_ssh_sg will be created
  + resource "aws_security_group" "web_ssh_sg" {
      + arn                    = (known after apply)
```

```
  + resource "aws_security_group" "web_ssh_sg" {
      + arn                    = (known after apply)
      + description            = "Allow inbound SSH on port 22 and HTTP on port 80"
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "Allow all outbound traffic"
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
              + self             = false
              + to_port          = 0
            },
        ]
      + id                     = (known after apply)
      + ingress                = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "HTTP from anywhere"
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
            },
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "SSH from anywhere"
              + from_port        = 22
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 22
            },
        ]
```

```
              + description      = "SSH from anywhere"
              + from_port        = 22
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 22
            },
        ]
      + name                   = "web-ssh-security-group"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags                   = {
          + "Name" = "web-ssh-sg"
        }
      + tags_all               = {
          + "Name" = "web-ssh-sg"
        }
      + vpc_id                 = "vpc-06214f99ff9b7aabe"
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_public_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

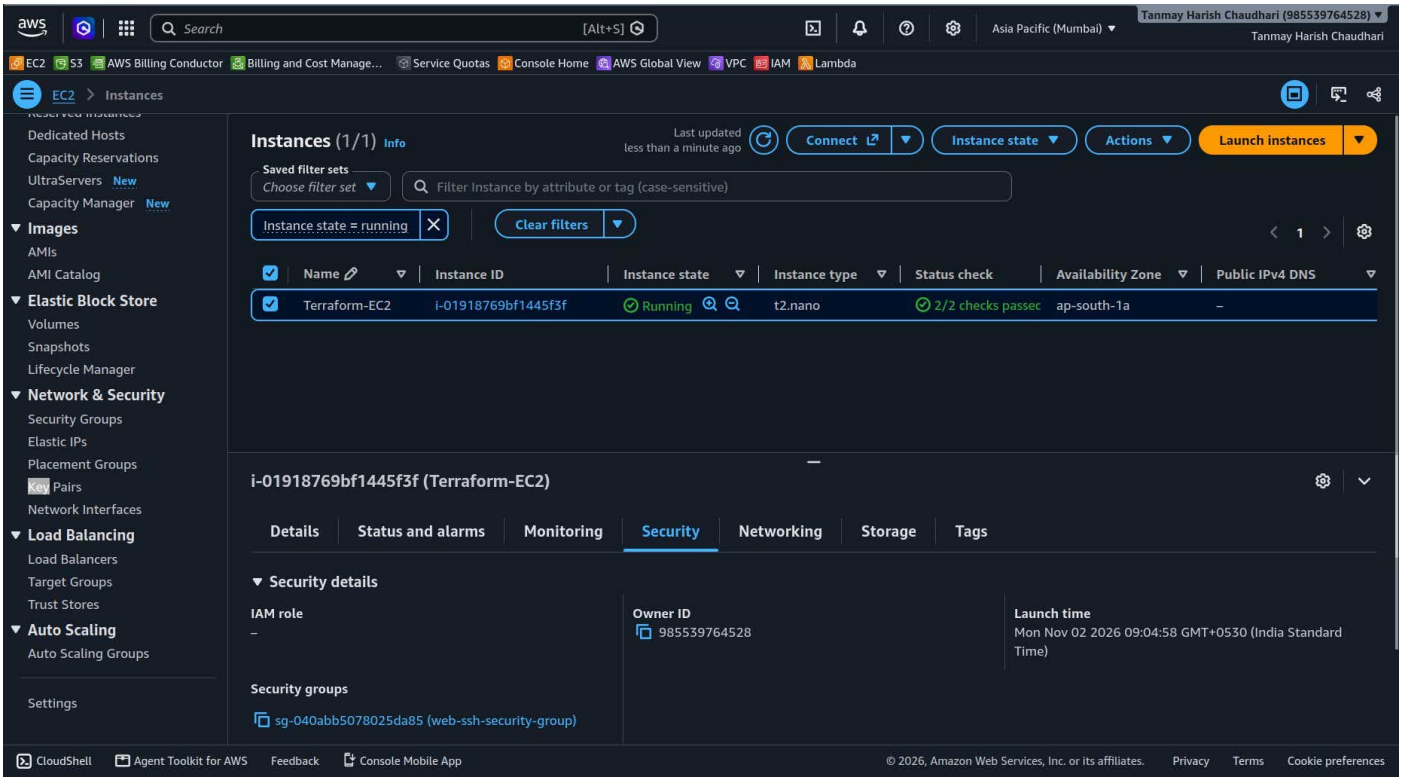
aws_security_group.web_ssh_sg: Creating...
aws_security_group.web_ssh_sg: Creation complete after 2s [id=sg-040abb5078025da85]
aws_instance.q1: Creating...
aws_instance.q1: Still creating... [00m10s elapsed]
aws_instance.q1: Creation complete after 13s [id=i-01918769bf1445f3f]

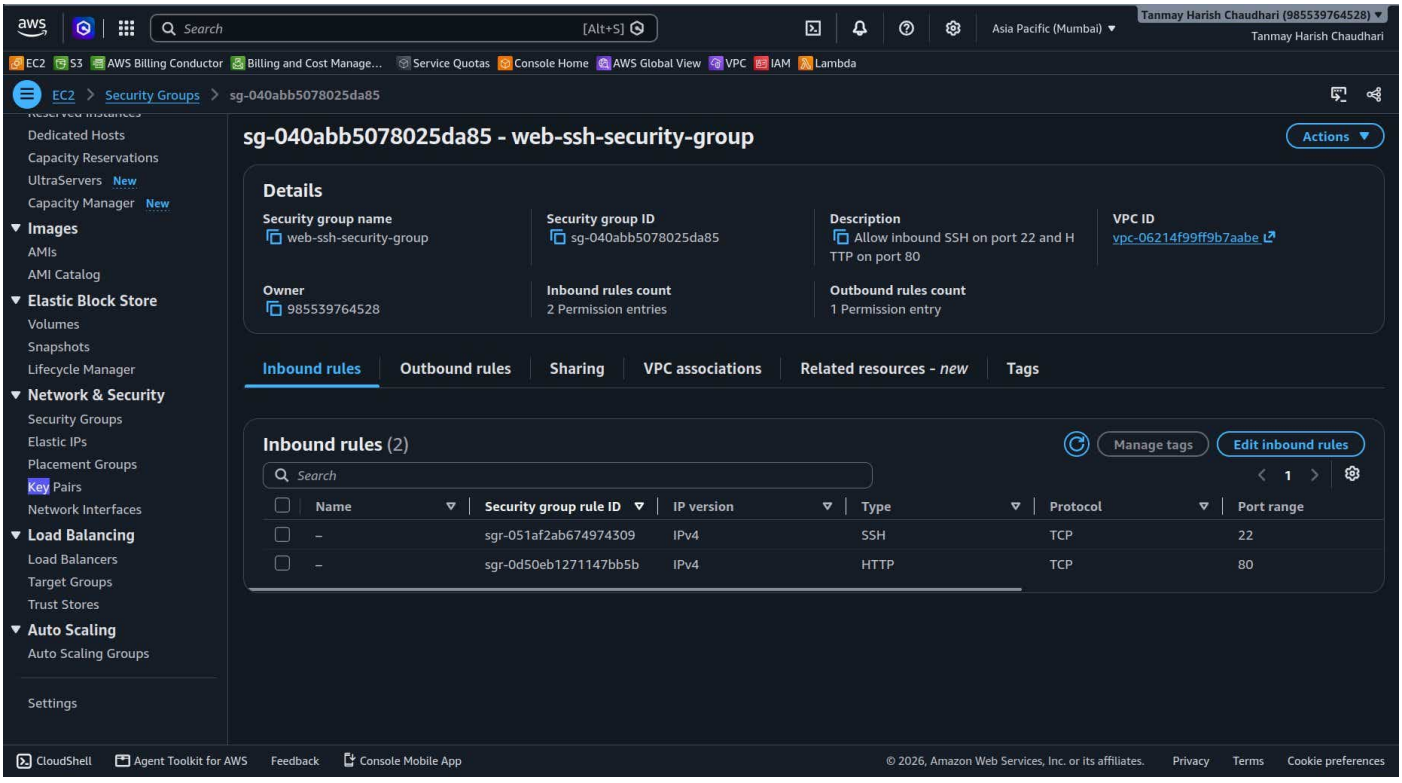
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

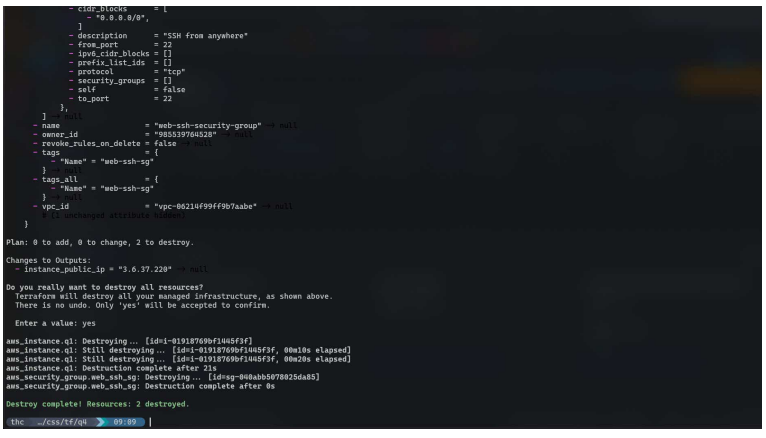
Outputs:

instance_public_ip = "3.6.37.220"

thc   …/css/tf/q4   09:05
```

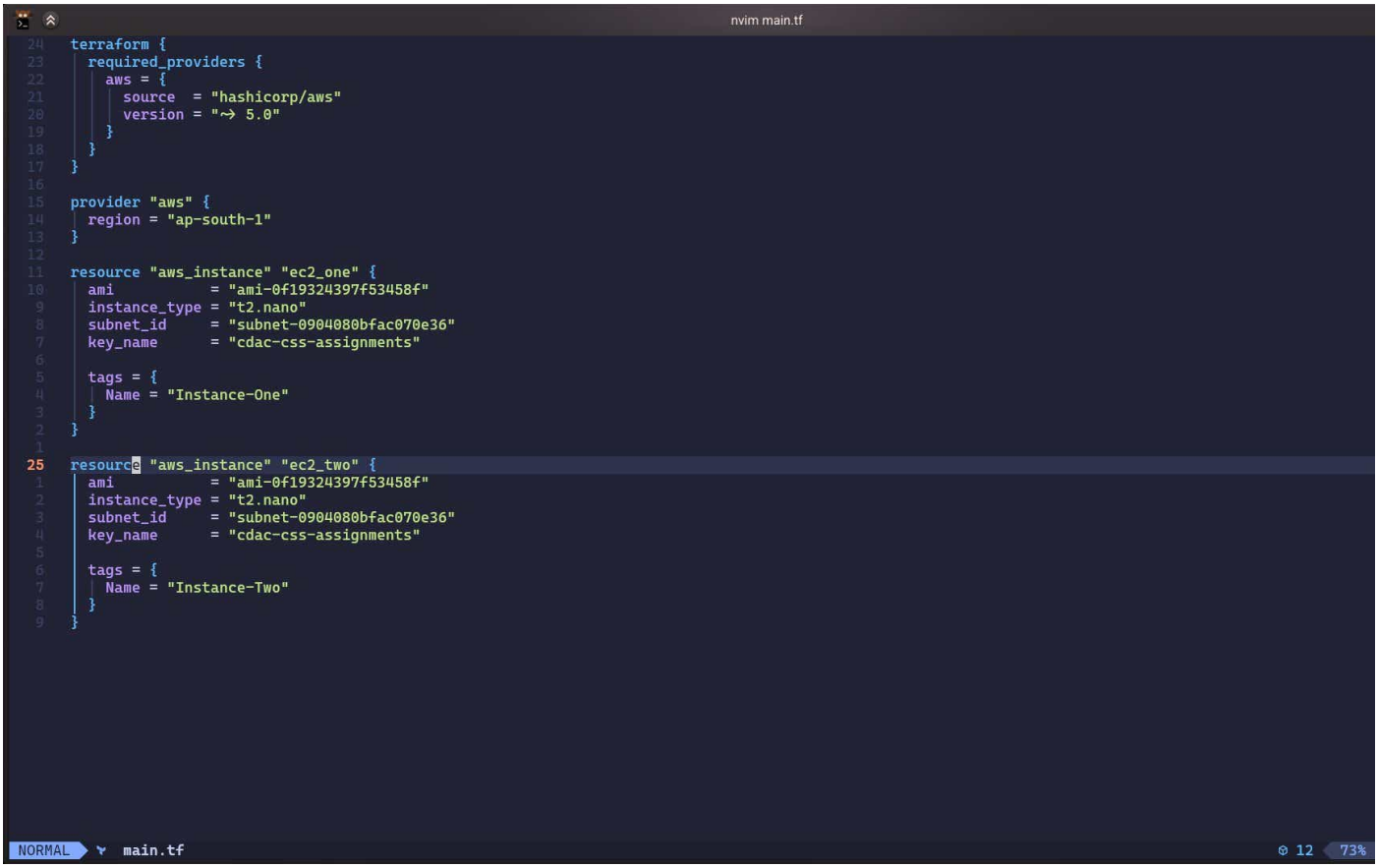

Search [Alt+S]
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
Tanmay Harish Chaudhari
EC2 S3 AWS Billing Conductor Billing and Cost Manage... Service Quotas Console Home AWS Global View VPC IAM Lambda
EC2 > Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Instances (1/1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Terraform-EC2
i-01918769bf1445f3f
Running
t2.nano
2/2 checks passec
ap-south-1a
i-01918769bf1445f3f (Terraform-EC2)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Security details
IAM role
Owner ID
985539764528
Launch time
Mon Nov 02 2026 09:04:58 GMT+0530 (India Standard Time)
Security groups
sg-040abb5078025da85 (web-ssh-security-group)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
Tanmay Harish Chaudhari
EC2 S3 AWS Billing Conductor Billing and Cost Manage... Service Quotas Console Home AWS Global View VPC IAM Lambda
EC2 > Security Groups > sg-040abb5078025da85
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
sg-040abb5078025da85 - web-ssh-security-group
Actions
Details
Security group name
web-ssh-security-group
Security group ID
sg-040abb5078025da85
Description
Allow inbound SSH on port 22 and HTTP on port 80
VPC ID
vpc-06214f99ff9b7aabe
Owner
985539764528
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Search
Name
Security group rule ID
IP version
Type
Protocol
Port range
sgr-051af2ab674974309
IPv4
SSH
TCP
22
sgr-0d50eb1271147bb5b
IPv4
HTTP
TCP
80
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


description = "SSH from anywhere"
from_port = 22
ipv6_cidr_blocks = []
prefix_list_ids = []
protocol = "tcp"
security_groups = []
self = false
to_port = 22
name = "web-ssh-security-group"
owner_id = "985539764528"
revoke_rules_on_delete = false
tags = {
"Name" = "web-ssh-sg"
tags_all = {
"Name" = "web-ssh-sg"
vpc_id = "vpc-06214f99ff9b7aabe"
Plan: 0 to add, 0 to change, 2 to destroy.
Changes to Outputs:
instance_public_ip = "3.6.37.220"
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
aws_instance.q1: Destroying... [id=i-01918769bf1445f3f]
aws_instance.q1: Still destroying... [id=i-01918769bf1445f3f, 00m10s elapsed]
aws_instance.q1: Still destroying... [id=i-01918769bf1445f3f, 00m20s elapsed]
aws_instance.q1: Destruction complete after 21s
aws_security_group.web_ssh_sg: Destroying... [id=sg-040abb5078025da85]
aws_security_group.web_ssh_sg: Destruction complete after 0s
Destroy complete! Resources: 2 destroyed.
thc ../css/tf/q4 09:09

Q.5: **Create two EC2 instances using Terraform and assign different names using tags.**

Output:




```
thc   …/css/tf/q5   09:24   terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc   …/css/tf/q5   09:24   terraform fmt

thc   …/css/tf/q5   09:25   terraform validate
Success! The configuration is valid.


thc   …/css/tf/q5   09:25   terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.ec2_one will be created
  + resource "aws_instance" "ec2_one" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```

```
    + disable_api_stop                     = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + enable_primary_ipv6                  = (known after apply)
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + host_resource_group_arn              = (known after apply)
    + iam_instance_profile                 = (known after apply)
    + id                                   = (known after apply)
    + instance_initiated_shutdown_behavior = (known after apply)
    + instance_lifecycle                   = (known after apply)
    + instance_state                       = (known after apply)
    + instance_type                        = "t2.nano"
    + ipv6_address_count                   = (known after apply)
    + ipv6_addresses                       = (known after apply)
    + key_name                             = "cdac-css-assignments"
    + monitoring                           = (known after apply)
    + outpost_arn                          = (known after apply)
    + password_data                        = (known after apply)
    + placement_group                      = (known after apply)
    + placement_partition_number           = (known after apply)
    + primary_network_interface_id         = (known after apply)
    + private_dns                          = (known after apply)
    + private_ip                           = (known after apply)
    + public_dns                           = (known after apply)
    + public_ip                            = (known after apply)
    + secondary_private_ips                = (known after apply)
    + security_groups                      = (known after apply)
    + source_dest_check                    = true
    + spot_instance_request_id             = (known after apply)
    + subnet_id                            = "subnet-0904080bfac070e36"
    + tags                                 = {
        + "Name" = "Instance-One"
      }
    + tags_all                             = {
        + "Name" = "Instance-One"
      }
    + tenancy                              = (known after apply)
    + user_data                            = (known after apply)
    + user_data_base64                     = (known after apply)
    + user_data_replace_on_change          = false
    + vpc_security_group_ids               = (known after apply)

    + capacity_reservation_specification (known after apply)

    + cpu_options (known after apply)
```

```
    + cpu_options (known after apply)

    + ebs_block_device (known after apply)

    + enclave_options (known after apply)

    + ephemeral_block_device (known after apply)

    + instance_market_options (known after apply)

    + maintenance_options (known after apply)

    + metadata_options (known after apply)

    + network_interface (known after apply)

    + private_dns_name_options (known after apply)

    + root_block_device (known after apply)
  }

# aws_instance.ec2_two will be created
+ resource "aws_instance" "ec2_two" {
    + ami                                  = "ami-0f19324397f53458f"
    + arn                                  = (known after apply)
    + associate_public_ip_address          = (known after apply)
    + availability_zone                    = (known after apply)
    + cpu_core_count                       = (known after apply)
    + cpu_threads_per_core                 = (known after apply)
    + disable_api_stop                     = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + enable_primary_ipv6                  = (known after apply)
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + host_resource_group_arn              = (known after apply)
    + iam_instance_profile                 = (known after apply)
    + id                                   = (known after apply)
    + instance_initiated_shutdown_behavior = (known after apply)
    + instance_lifecycle                   = (known after apply)
    + instance_state                       = (known after apply)
    + instance_type                        = "t2.nano"
    + ipv6_address_count                   = (known after apply)
    + ipv6_addresses                       = (known after apply)
    + key_name                             = "cdac-css-assignments"
    + monitoring                           = (known after apply)
    + outpost_arn                          = (known after apply)
```

```
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "Instance-Two"
        }
      + tags_all                             = {
          + "Name" = "Instance-Two"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)
```

```
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

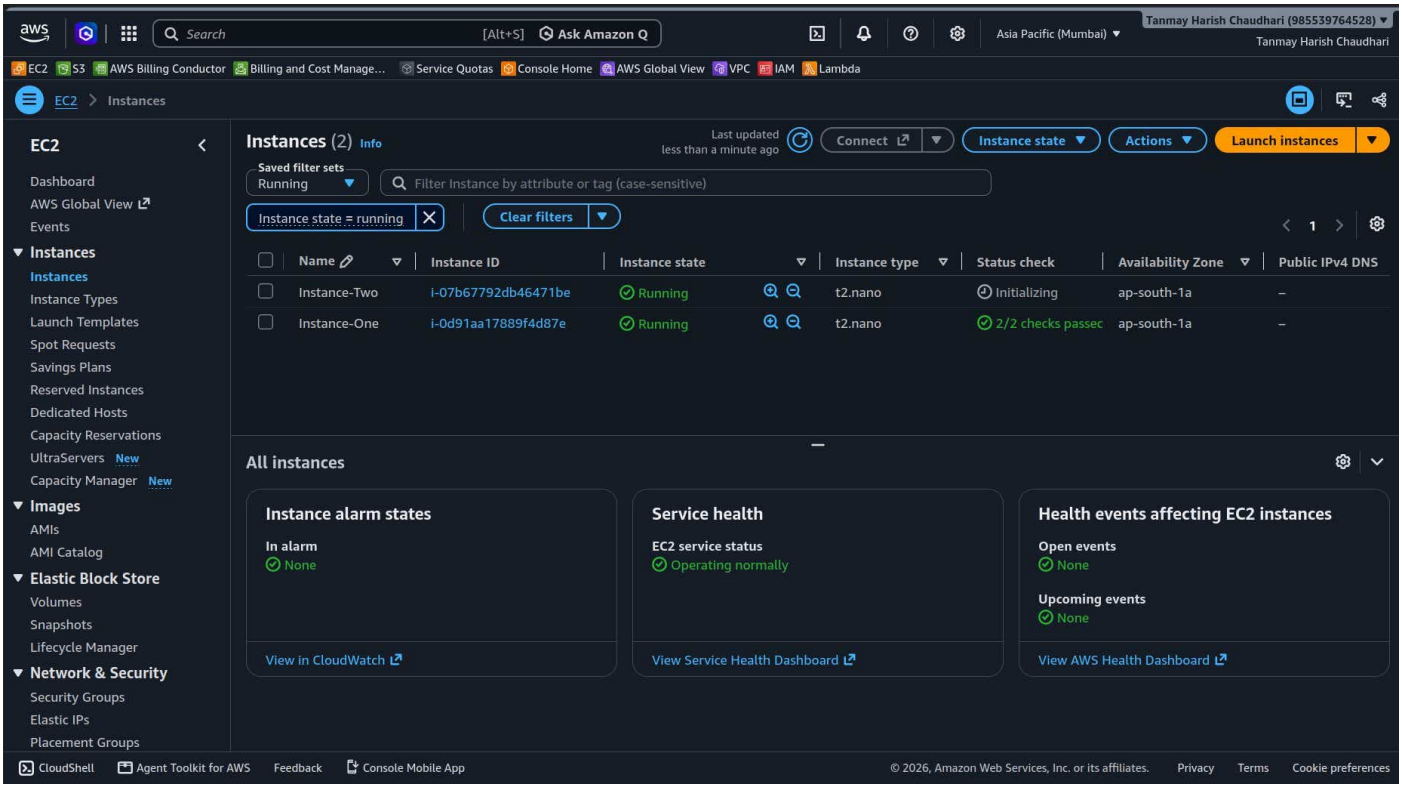
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.ec2_two: Creating...
aws_instance.ec2_one: Creating...
aws_instance.ec2_one: Still creating... [00m10s elapsed]
aws_instance.ec2_two: Still creating... [00m10s elapsed]
aws_instance.ec2_two: Creation complete after 17s [id=i-07b67792db46471be]
aws_instance.ec2_one: Creation complete after 17s [id=i-0d91aa17889f4d87e]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

thc   …/css/tf/q5   09:25
```


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
Tanmay Harish Chaudhari
EC2
S3
AWS Billing Conductor
Billing and Cost Manage...
Service Quotas
Console Home
AWS Global View
VPC
IAM
Lambda
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Instances (2) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Instance-Two
i-07b67792db46471be
Running
t2.nano
Initializing
ap-south-1a
–
Instance-One
i-0d91aa17889f4d87e
Running
t2.nano
2/2 checks passec
ap-south-1a
–
All instances
Instance alarm states
In alarm
None
View in CloudWatch
Service health
EC2 service status
Operating normally
View Service Health Dashboard
Health events affecting EC2 instances
Open events
None
Upcoming events
None
View AWS Health Dashboard
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Q.6: **Create an S3 bucket with versioning enabled using Terraform.**

Output:

```hcl
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "versioned_bucket" {
  bucket = "260840127026-tc-sample-file-versioned-bucket-css-assignment"
}

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.versioned_bucket.id

  versioning_configuration {
    status = "Enabled"
  }
}

output "bucket_name" {
  value = aws_s3_bucket.versioned_bucket.id
}
```

```
thc …/css/tf/q6 09:28 nvim main.tf

thc …/css/tf/q6 09:30 terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc …/css/tf/q6 09:30 terraform fmt

thc …/css/tf/q6 09:31 terraform validate
Success! The configuration is valid.


thc …/css/tf/q6 09:31 terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.versioned_bucket will be created
  + resource "aws_s3_bucket" "versioned_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "260840127026-tc-sample-file-versioned-bucket-css-assignment"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
```

```
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = (known after apply)
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

  # aws_s3_bucket_versioning.versioning_example will be created
  + resource "aws_s3_bucket_versioning" "versioning_example" {
      + bucket = (known after apply)
      + id     = (known after apply)

      + versioning_configuration {
          + mfa_delete = (known after apply)
          + status     = "Enabled"
        }
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + bucket_name = (known after apply)
```

```
Changes to Outputs:
  + bucket_name = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.versioned_bucket: Creating...
aws_s3_bucket.versioned_bucket: Creation complete after 2s [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]
aws_s3_bucket_versioning.versioning_example: Creating...
aws_s3_bucket_versioning.versioning_example: Creation complete after 2s [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

bucket_name = "260840127026-tc-sample-file-versioned-bucket-css-assignment"

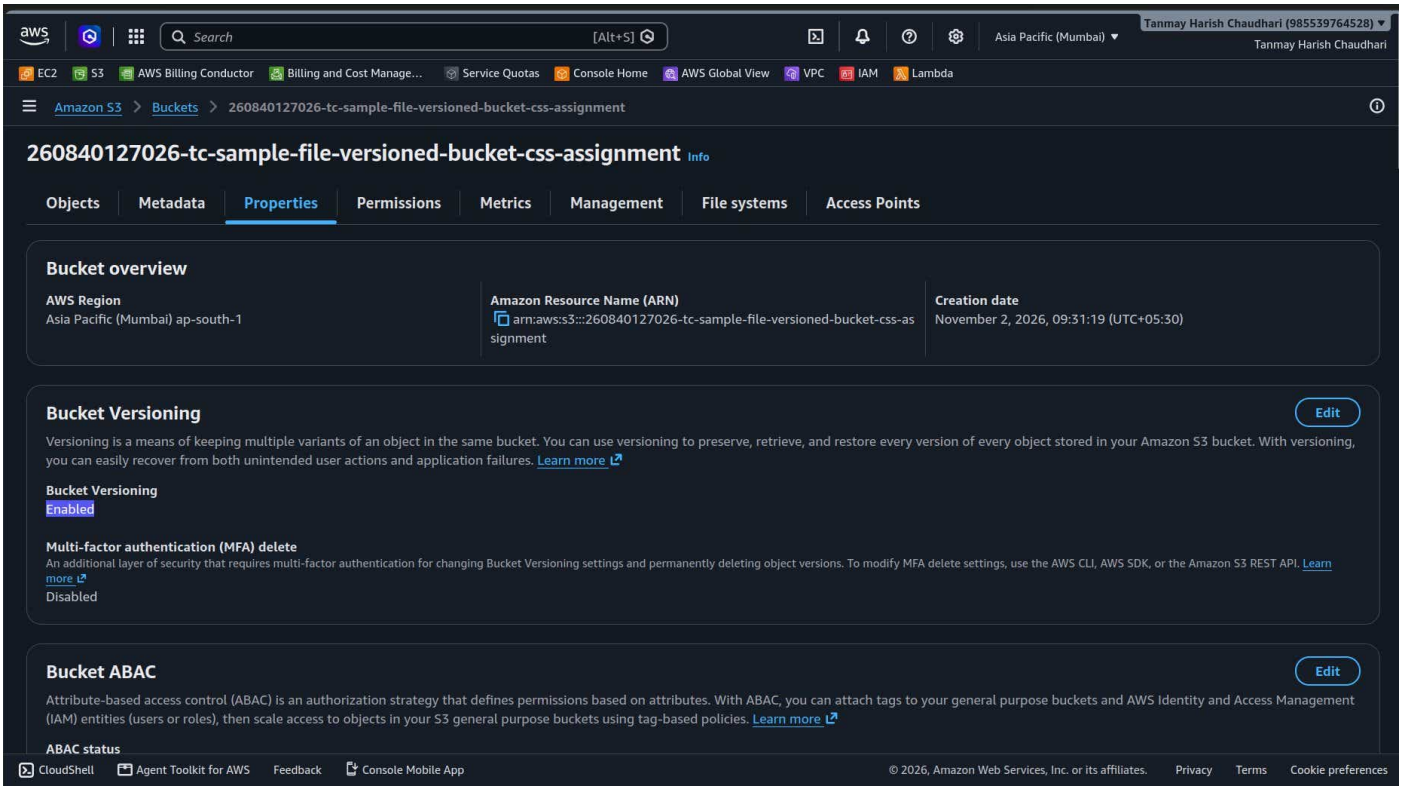
thc   …/css/tf/q6   09:31  terraform destroy
aws_s3_bucket.versioned_bucket: Refreshing state... [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]
aws_s3_bucket_versioning.versioning_example: Refreshing state... [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_s3_bucket.versioned_bucket will be destroyed
  - resource "aws_s3_bucket" "versioned_bucket" {
      - arn                         = "arn:aws:s3:::260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null
      - bucket                      = "260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null
      - bucket_domain_name          = "260840127026-tc-sample-file-versioned-bucket-css-assignment.s3.amazonaws.com" -> null
      - bucket_regional_domain_name = "260840127026-tc-sample-file-versioned-bucket-css-assignment.s3.ap-south-1.amazonaws.com" -> null
      - force_destroy               = false -> null
      - hosted_zone_id              = "Z11RGJOFQNVJUP" -> null
      - id                          = "260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null
      - object_lock_enabled         = false -> null
      - region                      = "ap-south-1" -> null
      - request_payer               = "BucketOwner" -> null
      - tags                        = {} -> null
      - tags_all                    = {} -> null
        # (3 unchanged attributes hidden)

      - grant {
          - id          = "c7fe4baf9606e65e43d9e898efefa2da342f5736353105073c006dcf7edbe704" -> null
```




```
        - rule {
            - bucket_key_enabled = false -> null

            - apply_server_side_encryption_by_default {
                - sse_algorithm     = "AES256" -> null
                # (1 unchanged attribute hidden)
              }
          }
      }

    - versioning {
        - enabled    = true -> null
        - mfa_delete = false -> null
      }
  }

# aws_s3_bucket_versioning.versioning_example will be destroyed
- resource "aws_s3_bucket_versioning" "versioning_example" {
    - bucket = "260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null
    - id     = "260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null
      # (1 unchanged attribute hidden)

    - versioning_configuration {
        - status     = "Enabled" -> null
          # (1 unchanged attribute hidden)
      }
  }

Plan: 0 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  - bucket_name = "260840127026-tc-sample-file-versioned-bucket-css-assignment" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_s3_bucket_versioning.versioning_example: Destroying... [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]
aws_s3_bucket_versioning.versioning_example: Destruction complete after 0s
aws_s3_bucket.versioned_bucket: Destroying... [id=260840127026-tc-sample-file-versioned-bucket-css-assignment]
aws_s3_bucket.versioned_bucket: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.

thc  …/css/tf/q6  09:32
```

Q.7: **Create an IAM group and an IAM user using Terraform, and add the user to the group.**

Output:

```
thc …/css/tf/q7 09:36 nvim main.tf

thc …/css/tf/q7 09:37 cat main.tf

File: main.tf

1  terraform {
2    required_providers {
3      aws = {
4        source  = "hashicorp/aws"
5        version = "~> 5.0"
6      }
7    }
8  }
9
10 provider "aws" {
11   region = "ap-south-1"
12 }
13
14 resource "aws_iam_group" "developers" {
15   name = "developers-group"
16 }
17
18 resource "aws_iam_user" "dev_user" {
19   name = "developer-one"
20 }
21
22 resource "aws_iam_group_membership" "dev_team" {
23   name  = "developer-group-membership"
24   users = [aws_iam_user.dev_user.name]
25   group = aws_iam_group.developers.name
26 }

thc …/css/tf/q7 09:37 terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

```
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc …/css/tf/q7 09:38 terraform validate
Success! The configuration is valid.


thc …/css/tf/q7 09:38 terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_group.developers will be created
  + resource "aws_iam_group" "developers" {
      + arn       = (known after apply)
      + id        = (known after apply)
      + name      = "developers-group"
      + path      = "/"
      + unique_id = (known after apply)
    }

  # aws_iam_group_membership.dev_team will be created
  + resource "aws_iam_group_membership" "dev_team" {
      + group = "developers-group"
      + id    = (known after apply)
      + name  = "developer-group-membership"
      + users = [
          + "developer-one",
        ]
    }

  # aws_iam_user.dev_user will be created
  + resource "aws_iam_user" "dev_user" {
      + arn           = (known after apply)
      + force_destroy = false
```

```
  + resource "aws_iam_group" "developers" {
      + arn       = (known after apply)
      + id        = (known after apply)
      + name      = "developers-group"
      + path      = "/"
      + unique_id = (known after apply)
    }

  # aws_iam_group_membership.dev_team will be created
  + resource "aws_iam_group_membership" "dev_team" {
      + group = "developers-group"
      + id    = (known after apply)
      + name  = "developer-group-membership"
      + users = [
          + "developer-one",
        ]
    }

  # aws_iam_user.dev_user will be created
  + resource "aws_iam_user" "dev_user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "developer-one"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.

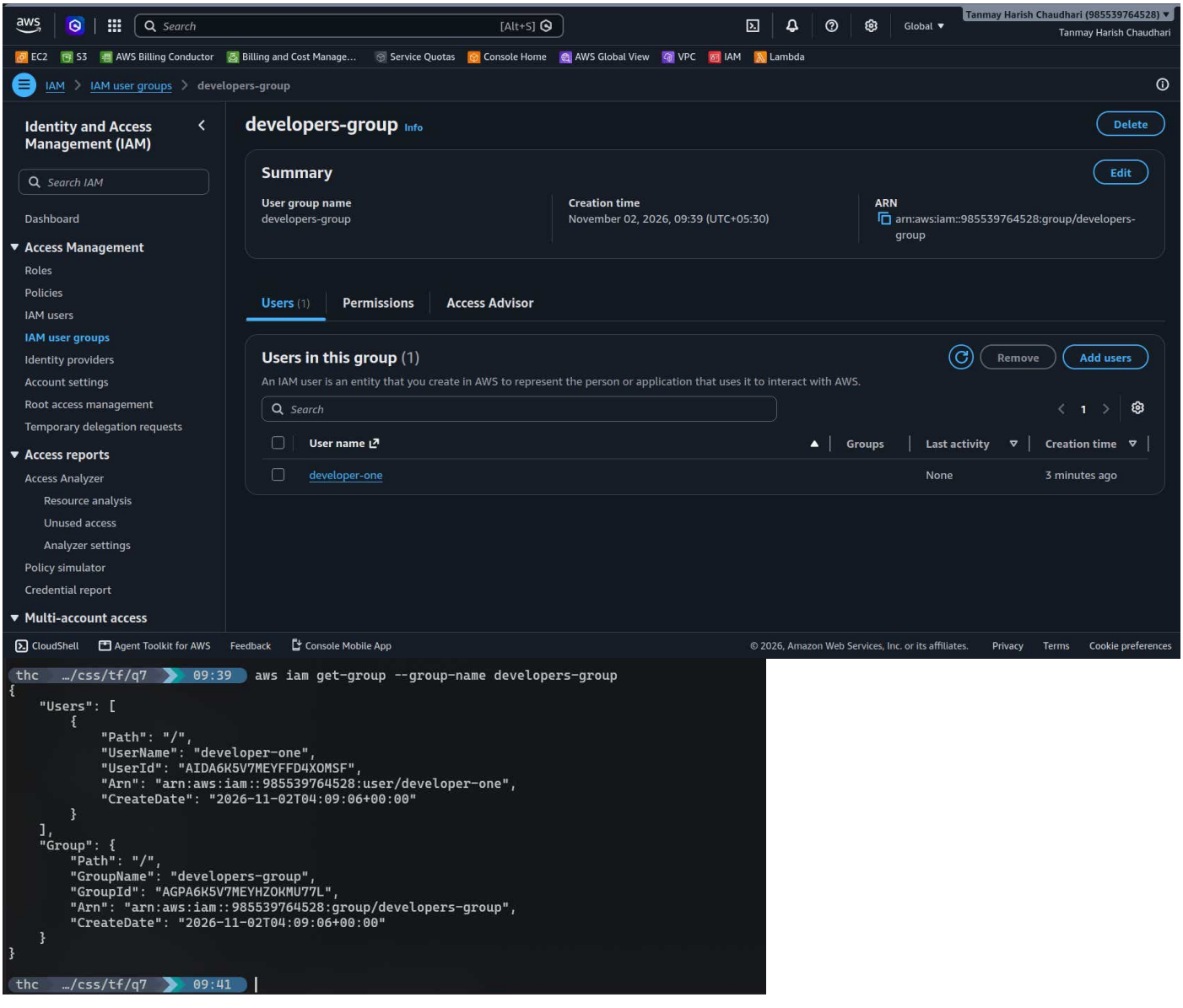
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_group.developers: Creating ...
aws_iam_user.dev_user: Creating ...
aws_iam_user.dev_user: Creation complete after 1s [id=developer-one]
aws_iam_group.developers: Creation complete after 2s [id=developers-group]
aws_iam_group_membership.dev_team: Creating ...
aws_iam_group_membership.dev_team: Creation complete after 0s [id=developer-group-membership]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

thc  …/css/tf/q7  09:39  1
```




```
thc  …/css/tf/q7  09:39  aws iam get-group --group-name developers-group
{
    "Users": [
        {
            "Path": "/",
            "UserName": "developer-one",
            "UserId": "AIDA6K5V7MEYFFD4XOMSF",
            "Arn": "arn:aws:iam::985539764528:user/developer-one",
            "CreateDate": "2026-11-02T04:09:06+00:00"
        }
    ],
    "Group": {
        "Path": "/",
        "GroupName": "developers-group",
        "GroupId": "AGPA6K5V7MEYHZOKMU77L",
        "Arn": "arn:aws:iam::985539764528:group/developers-group",
        "CreateDate": "2026-11-02T04:09:06+00:00"
    }
}

thc  …/css/tf/q7  09:41
```

Q.8: **Create an EC2 instance and associate an IAM role with it to allow access to an S3 bucket.**

Output:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}


provider "aws" {
  region = "ap-south-1"
}


resource "aws_iam_role" "ec2_s3_access_role" {
  name = "ec2-s3-access-role"


  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
      }
    ]
  })
}


resource "aws_iam_role_policy_attachment" "s3_access_attach" {
  role       = aws_iam_role.ec2_s3_access_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
}


resource "aws_iam_instance_profile" "ec2_s3_profile" {
  name = "ec2-s3-instance-profile"
  role = aws_iam_role.ec2_s3_access_role.name
}


resource "aws_instance" "app_server" {
  ami                  = "ami-0f19324397f53458f"
  instance_type        = "t2.nano"
  subnet_id            = "subnet-0904080bfac070e36"
  key_name             = "cdac-css-assignments"
  iam_instance_profile = aws_iam_instance_profile.ec2_s3_profile.name


  tags = {
    Name = "EC2-S3-Role-Instance"
  }
}


output "instance_public_ip" {
  value = aws_instance.app_server.public_ip
```

}

```
thc   ~/css/tf    09:44  cd q8
./  ../

thc   …/css/tf/q8    09:44  nvim main.tf

thc   …/css/tf/q8    09:48  nvim main.tf

thc   …/css/tf/q8    09:48  terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc   …/css/tf/q8    09:49  terraform validate
Success! The configuration is valid.


thc   …/css/tf/q8    09:49  terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_instance_profile.ec2_s3_profile will be created
  + resource "aws_iam_instance_profile" "ec2_s3_profile" {
      + arn         = (known after apply)
      + create_date = (known after apply)
      + id          = (known after apply)
      + name        = "ec2-s3-instance-profile"
```

```
      + name        = "ec2-s3-instance-profile"
      + name_prefix = (known after apply)
      + path        = "/"
      + role        = "ec2-s3-access-role"
      + tags_all    = (known after apply)
      + unique_id   = (known after apply)
    }

  # aws_iam_role.ec2_s3_access_role will be created
  + resource "aws_iam_role" "ec2_s3_access_role" {
      + arn                   = (known after apply)
      + assume_role_policy    = jsonencode(
            {
              + Statement = [
                  + {
                      + Action    = "sts:AssumeRole"
                      + Effect    = "Allow"
                      + Principal = {
                          + Service = "ec2.amazonaws.com"
                        }
                    },
                ]
              + Version   = "2012-10-17"
            }
        )
      + create_date           = (known after apply)
      + force_detach_policies = false
      + id                    = (known after apply)
      + managed_policy_arns   = (known after apply)
      + max_session_duration  = 3600
      + name                  = "ec2-s3-access-role"
      + name_prefix           = (known after apply)
      + path                  = "/"
      + tags_all              = (known after apply)
      + unique_id             = (known after apply)

      + inline_policy (known after apply)
    }

  # aws_iam_role_policy_attachment.s3_access_attach will be created
  + resource "aws_iam_role_policy_attachment" "s3_access_attach" {
      + id         = (known after apply)
      + policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
      + role       = "ec2-s3-access-role"
    }

  # aws_instance.app_server will be created
```

```
  # aws_instance.app_server will be created
  + resource "aws_instance" "app_server" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = "ec2-s3-instance-profile"
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "EC2-S3-Role-Instance"
        }
      + tags_all                             = {
          + "Name" = "EC2-S3-Role-Instance"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
```

```
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_public_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_role.ec2_s3_access_role: Creating...
aws_iam_role.ec2_s3_access_role: Creation complete after 3s [id=ec2-s3-access-role]
aws_iam_role_policy_attachment.s3_access_attach: Creating...
aws_iam_instance_profile.ec2_s3_profile: Creating...
aws_iam_role_policy_attachment.s3_access_attach: Creation complete after 1s [id=ec2-s3-access-role-20261102041955846600000001]
aws_iam_instance_profile.ec2_s3_profile: Creation complete after 9s [id=ec2-s3-instance-profile]
aws_instance.app_server: Creating...
```

```
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_role.ec2_s3_access_role: Creating...
aws_iam_role.ec2_s3_access_role: Creation complete after 3s [id=ec2-s3-access-role]
aws_iam_role_policy_attachment.s3_access_attach: Creating...
aws_iam_instance_profile.ec2_s3_profile: Creating...
aws_iam_role_policy_attachment.s3_access_attach: Creation complete after 1s [id=ec2-s3-access-role-20261102041955846600000001]
aws_iam_instance_profile.ec2_s3_profile: Creation complete after 9s [id=ec2-s3-instance-profile]
aws_instance.app_server: Creating...
aws_instance.app_server: Still creating... [00m10s elapsed]
aws_instance.app_server: Creation complete after 17s [id=i-048a9f5f5856e6e6a]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

instance_public_ip = "13.206.89.243"

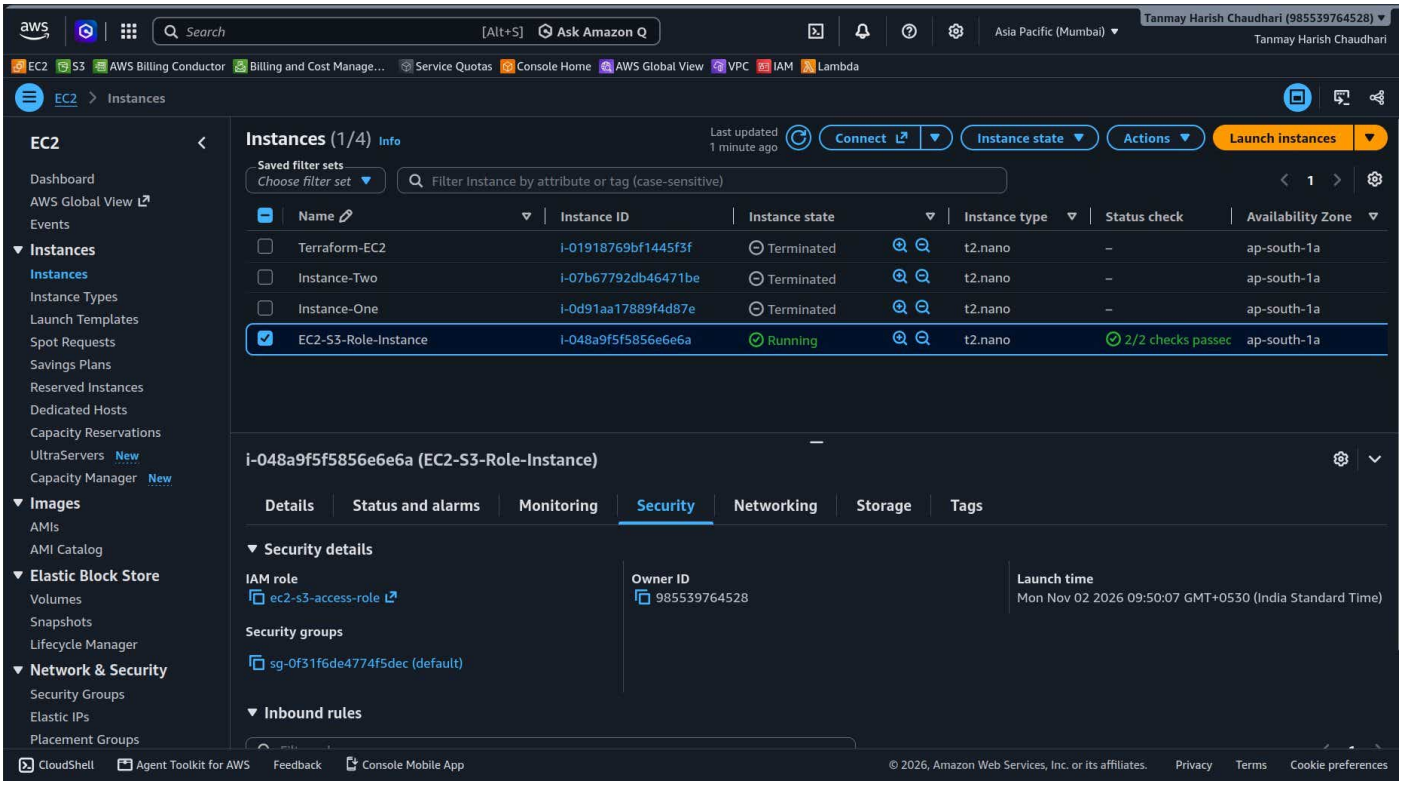
thc  …/css/tf/q8  09:50  nvim main.tf

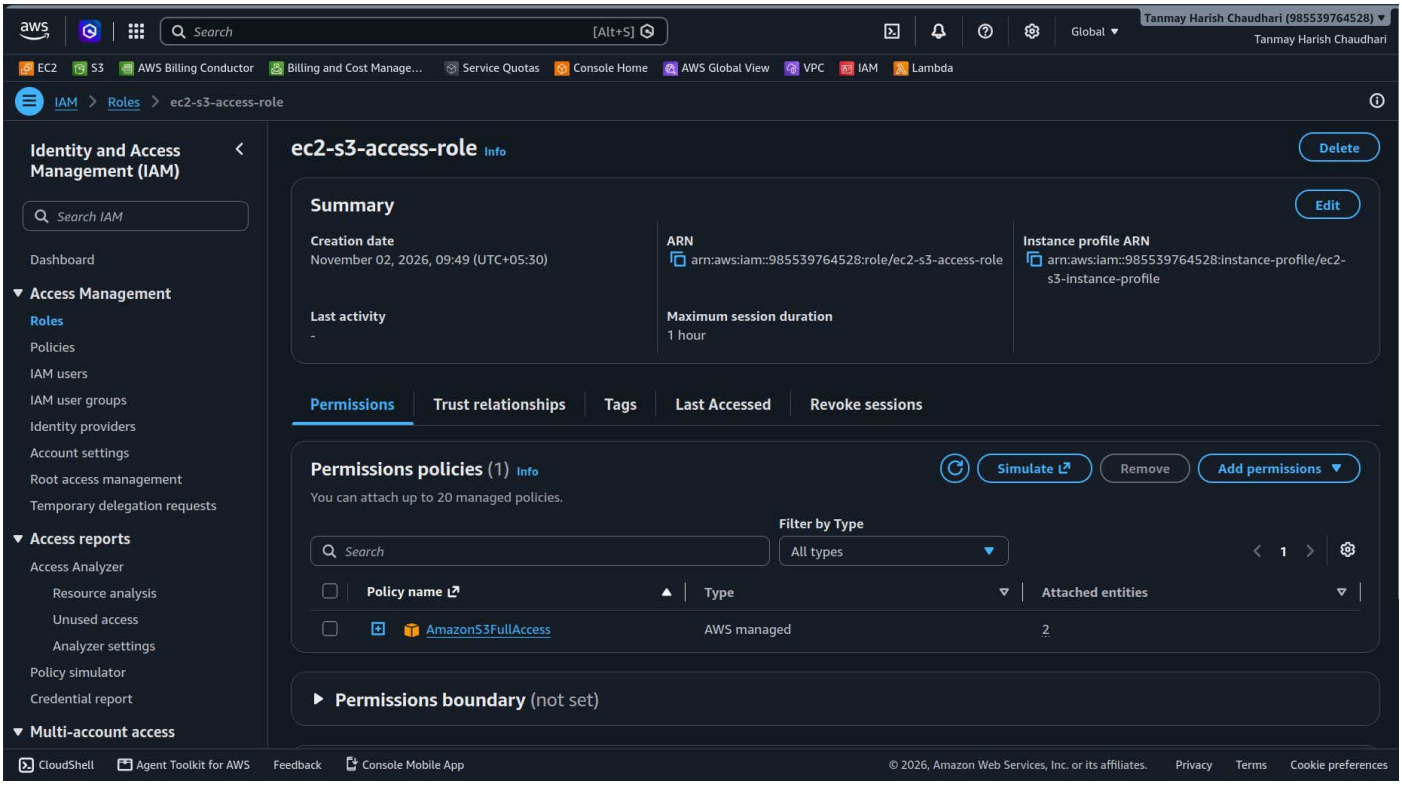
thc  …/css/tf/q8  10:00  terraform destroy
aws_iam_role.ec2_s3_access_role: Refreshing state... [id=ec2-s3-access-role]
aws_iam_role_policy_attachment.s3_access_attach: Refreshing state... [id=ec2-s3-access-role-20261102041955846600000001]
aws_iam_instance_profile.ec2_s3_profile: Refreshing state... [id=ec2-s3-instance-profile]
aws_instance.app_server: Refreshing state... [id=i-048a9f5f5856e6e6a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_iam_instance_profile.ec2_s3_profile will be destroyed
  - resource "aws_iam_instance_profile" "ec2_s3_profile" {
      - arn         = "arn:aws:iam::985539764528:instance-profile/ec2-s3-instance-profile" -> null
      - create_date = "2026-11-02T04:19:56Z" -> null
      - id          = "ec2-s3-instance-profile" -> null
      - name        = "ec2-s3-instance-profile" -> null
      - path        = "/" -> null
      - role        = "ec2-s3-access-role" -> null
      - tags        = {} -> null
      - tags_all    = {} -> null
      - unique_id   = "AIPA6K5V7MEYCJOKVS5M6" -> null
        # (1 unchanged attribute hidden)
    }
```




Search
[Alt+S]
Global
Tanmay Harish Chaudhari (985539764528)
Tanmay Harish Chaudhari
EC2
S3
AWS Billing Conductor
Billing and Cost Manage...
Service Quotas
Console Home
AWS Global View
VPC
IAM
Lambda
IAM > Roles > ec2-s3-access-role
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Multi-account access
ec2-s3-access-role Info
Delete
Summary
Edit
Creation date
November 02, 2026, 09:49 (UTC+05:30)
ARN
arn:aws:iam::985539764528:role/ec2-s3-access-role
Instance profile ARN
arn:aws:iam::985539764528:instance-profile/ec2-s3-instance-profile
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (1) Info
You can attach up to 20 managed policies.
Simulate
Remove
Add permissions
Filter by Type
Search
All types
1
Policy name
Type
Attached entities
AmazonS3FullAccess
AWS managed
2
Permissions boundary (not set)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Q.9: **Use Terraform variables to specify the EC2 instance type, S3 bucket name, and AWS region..**

Output:

```
thc  ~/css/tf  10:17  mkdir q9

thc  ~/css/tf  10:17  cd q9
./  ../

thc  …/css/tf/q9  10:17  nvim main.tf

thc  …/css/tf/q9  10:17  cat main.tf
```

File: main.tf

```hcl
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "sample_bucket" {
  bucket = var.s3_bucket_name
}

resource "aws_instance" "q1" {
  ami           = "ami-0f19324397f53458f"
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
  key_name      = var.key_name

  tags = {
    Name = "Terraform-EC2"
  }
}

output "s3_bucket_id" {
  value = aws_s3_bucket.sample_bucket.id
}

output "instance_id" {
  value = aws_instance.q1.id
}
```

```
thc  …/css/tf/q9  10:17  nvim variables.tf

thc  …/css/tf/q9  10:18  cat variables.tf
```

File: variables.tf

```hcl
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.nano"
}

variable "s3_bucket_name" {
  description = "unique name for the S3 bucket"
  type        = string
}

variable "subnet_id" {
  description = "Subnet ID for the EC2 instance"
  type        = string
}

variable "key_name" {
  description = "Existing key pair name for SSH access"
  type        = string
}
```

```
thc  …/css/tf/q9  10:18  nvim terraform.tfvars

thc  …/css/tf/q9  10:19  cat terraform.tfvars
```

File: terraform.tfvars

```
aws_region     = "ap-south-1"
instance_type  = "t2.nano"
s3_bucket_name = "260840127026-tc-sample-file-bucket-css-assignment"
subnet_id      = "subnet-0904080bfac070e36"
key_name       = "cdac-css-assignments"
```

```
  4 | subnet_id       = "subnet-0904080bfac070e36"
  5 | key_name        = "cdac-css-assignments"

thc   …/css/tf/q9   10:19  terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc   …/css/tf/q9   10:20  terraform validate
Success! The configuration is valid.


thc   …/css/tf/q9   10:21  terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.q1 will be created
  + resource "aws_instance" "q1" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
```

```
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "Terraform-EC2"
        }
      + tags_all                             = {
          + "Name" = "Terraform-EC2"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)
```

```
      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

  # aws_s3_bucket.sample_bucket will be created
  + resource "aws_s3_bucket" "sample_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = (known after apply)
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)
```

```
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id  = (known after apply)
  + s3_bucket_id = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

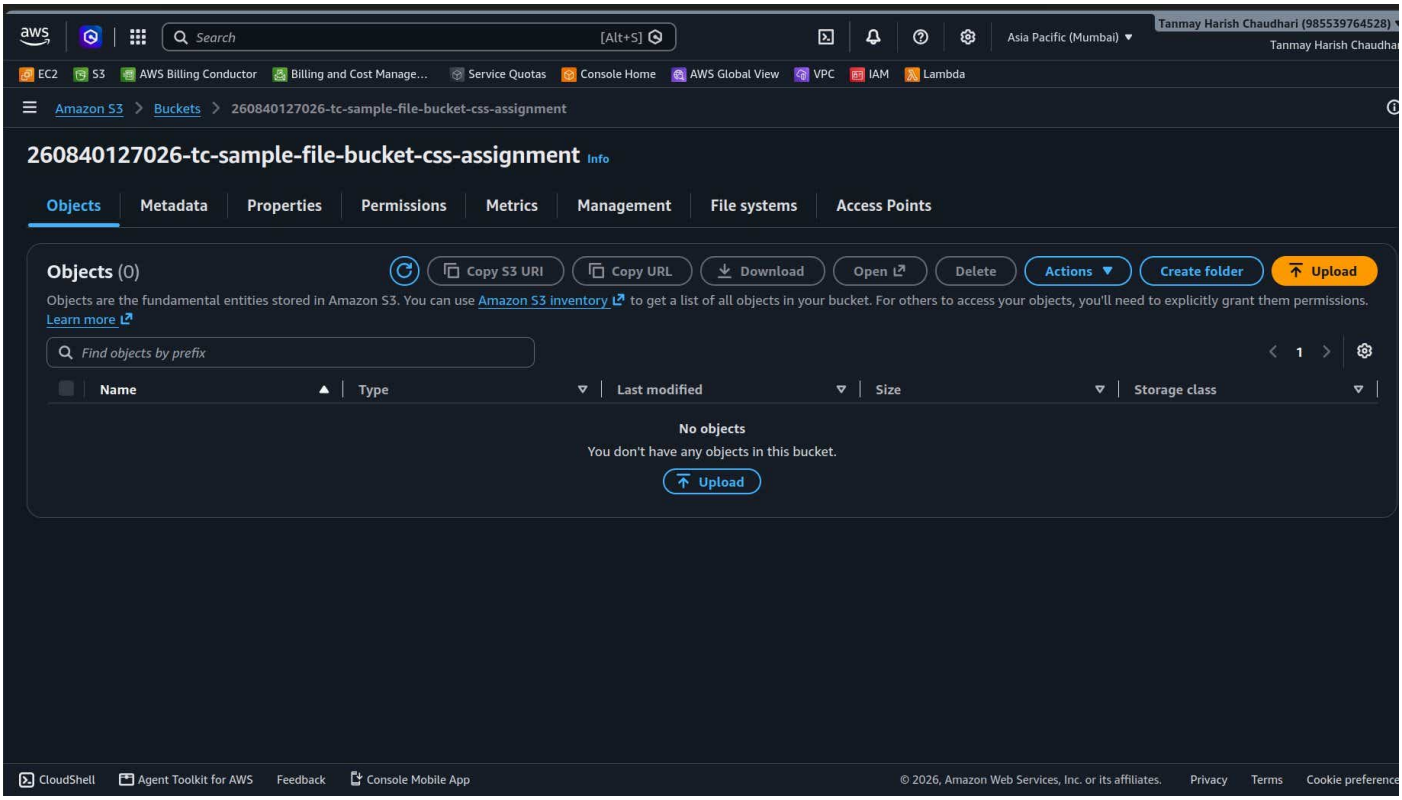
aws_s3_bucket.sample_bucket: Creating...
aws_instance.q1: Creating...
aws_s3_bucket.sample_bucket: Creation complete after 2s [id=260840127026-tc-sample-file-bucket-css-assignment]
aws_instance.q1: Still creating... [00m10s elapsed]
aws_instance.q1: Creation complete after 13s [id=i-03989840ada44b769]

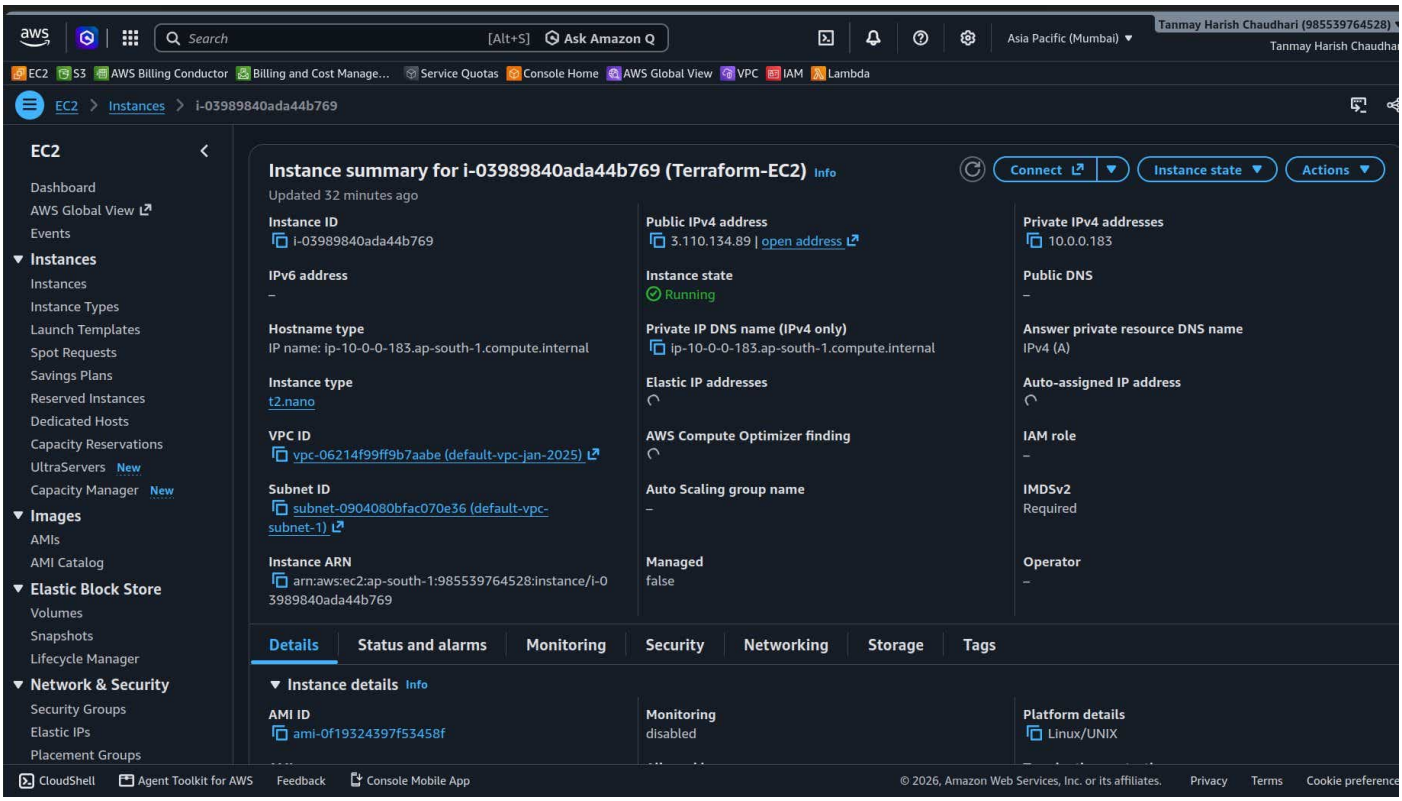
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-03989840ada44b769"
s3_bucket_id = "260840127026-tc-sample-file-bucket-css-assignment"

thc   …/css/tf/q9   10:21
```


Search
[Alt+S]
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
EC2
S3
AWS Billing Conductor
Billing and Cost Manage...
Service Quotas
Console Home
AWS Global View
VPC
IAM
Lambda
Amazon S3 > Buckets > 260840127026-tc-sample-file-bucket-css-assignment
260840127026-tc-sample-file-bucket-css-assignment Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Objects (0)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions.
Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
No objects
You don't have any objects in this bucket.
Upload
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preference


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Tanmay Harish Chaudhari (985539764528)
EC2 > Instances > i-03989840ada44b769
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Instance summary for i-03989840ada44b769 (Terraform-EC2) Info
Updated 32 minutes ago
Connect
Instance state
Actions
Instance ID
i-03989840ada44b769
IPv6 address
–
Hostname type
IP name: ip-10-0-0-183.ap-south-1.compute.internal
Instance type
t2.nano
VPC ID
vpc-06214f99ff9b7aabe (default-vpc-jan-2025)
Subnet ID
subnet-0904080bfac070e36 (default-vpc-subnet-1)
Instance ARN
arn:aws:ec2:ap-south-1:985539764528:instance/i-03989840ada44b769
Public IPv4 address
3.110.134.89 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-10-0-0-183.ap-south-1.compute.internal
Elastic IP addresses
AWS Compute Optimizer finding
Auto Scaling group name
–
Managed
false
Private IPv4 addresses
10.0.0.183
Public DNS
–
Answer private resource DNS name
IPv4 (A)
Auto-assigned IP address
IAM role
–
IMDSv2
Required
Operator
–
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance details Info
AMI ID
ami-0f19324397f53458f
Monitoring
disabled
Platform details
Linux/UNIX
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preference

Q.10: **Use Terraform to create an EC2 instance and an S3 bucket. Then modify the configuration to change the instance type and apply the changes.**

Output:

Initial Configuration:

```hcl
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "260840127026-tc-sample-file-bucket-css-assignment"
}

resource "aws_instance" "my_server" {
  ami           = "ami-0f19324397f53458f"
  instance_type = "t2.nano"
  subnet_id     = "subnet-0904080bfac070e36"
  key_name      = "cdac-css-assignments"

  tags = {
    Name = "Terraform-Modifiable-EC2"
  }
}

output "instance_id" {
  value = aws_instance.my_server.id
}

output "current_instance_type" {
  value = aws_instance.my_server.instance_type
}

output "bucket_name" {
  value = aws_s3_bucket.my_bucket.id
}
```

NORMAL main.tf 1 12 76% 30:35

```
thc …/css/tf/q10 10:29 nvim main.tf

thc …/css/tf/q10 10:29 terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

thc …/css/tf/q10 10:29 terraform validate
Success! The configuration is valid.


thc …/css/tf/q10 10:30 terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.my_server will be created
  + resource "aws_instance" "my_server" {
      + ami                                  = "ami-0f19324397f53458f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```

```
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.nano"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "cdac-css-assignments"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0904080bfac070e36"
      + tags                                 = {
          + "Name" = "Terraform-Modifiable-EC2"
        }
      + tags_all                             = {
          + "Name" = "Terraform-Modifiable-EC2"
        }
      + tenancy                              = (known after apply)
      + user_data                            = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)
```

```
      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

  # aws_s3_bucket.my_bucket will be created
  + resource "aws_s3_bucket" "my_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "260840127026-tc-sample-file-bucket-css-assignment"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = (known after apply)
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)
```

```
      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + bucket_name          = (known after apply)
  + current_instance_type = "t2.nano"
  + instance_id          = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.my_bucket: Creating...
aws_instance.my_server: Creating...
aws_s3_bucket.my_bucket: Creation complete after 2s [id=260840127026-tc-sample-file-bucket-css-assignment]
aws_instance.my_server: Still creating... [00m10s elapsed]
aws_instance.my_server: Creation complete after 12s [id=i-0a27a9f784a88cd40]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

bucket_name = "260840127026-tc-sample-file-bucket-css-assignment"
current_instance_type = "t2.nano"
instance_id = "i-0a27a9f784a88cd40"

thc   …/css/tf/q10   10:30

thc   …/css/tf/q10   10:30

thc   …/css/tf/q10   10:30
```

After modifying instance type from t2.nano --> t2.micro

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "260840127026-tc-sample-file-bucket-css-assignment"
}

resource "aws_instance" "my_server" {
  ami           = "ami-0f19324397f53458f"
  instance_type = "t2.micro"
  subnet_id     = "subnet-0904080bfac070e36"
  key_name      = "cdac-css-assignments"

  tags = {
    Name = "Terraform-Modifiable-EC2"
  }
}

output "instance_id" {
  value = aws_instance.my_server.id
}

output "current_instance_type" {
  value = aws_instance.my_server.instance_type
}

output "bucket_name" {
  value = aws_s3_bucket.my_bucket.id
}

VISUAL  main.tf                                                    29 < ⚙ 12  51%  20:1
```

```
thc   …/css/tf/q10  10:30  nvim main.tf

thc   …/css/tf/q10  10:31

thc   …/css/tf/q10  10:31  terraform plan
aws_s3_bucket.my_bucket: Refreshing state... [id=260840127026-tc-sample-file-bucket-css-assignment]
aws_instance.my_server: Refreshing state... [id=i-0a27a9f784a88cd40]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.my_server will be updated in-place
  ~ resource "aws_instance" "my_server" {
        id                                   = "i-0a27a9f784a88cd40"
      ~ instance_type                        = "t2.nano" -> "t2.micro"
      + public_dns                           = (known after apply)
      ~ public_ip                            = "13.233.3.63" -> (known after apply)
        tags                                 = {
            "Name" = "Terraform-Modifiable-EC2"
        }
        # (36 unchanged attributes hidden)

        # (8 unchanged blocks hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.

Changes to Outputs:
  ~ current_instance_type = "t2.nano" -> "t2.micro"

---

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

thc   …/css/tf/q10  10:31  terraform apply
aws_s3_bucket.my_bucket: Refreshing state... [id=260840127026-tc-sample-file-bucket-css-assignment]
aws_instance.my_server: Refreshing state... [id=i-0a27a9f784a88cd40]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.my_server will be updated in-place
  ~ resource "aws_instance" "my_server" {
```

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.my_server will be updated in-place
  ~ resource "aws_instance" "my_server" {
        id                                   = "i-0a27a9f784a88cd40"
      ~ instance_type                        = "t2.nano" -> "t2.micro"
      + public_dns                           = (known after apply)
      ~ public_ip                            = "13.233.3.63" -> (known after apply)
        tags                                 = {
            "Name" = "Terraform-Modifiable-EC2"
        }
        # (36 unchanged attributes hidden)

        # (8 unchanged blocks hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.

Changes to Outputs:
  ~ current_instance_type = "t2.nano" -> "t2.micro"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.my_server: Modifying... [id=i-0a27a9f784a88cd40]
aws_instance.my_server: Still modifying... [id=i-0a27a9f784a88cd40, 00m10s elapsed]
aws_instance.my_server: Still modifying... [id=i-0a27a9f784a88cd40, 00m20s elapsed]
aws_instance.my_server: Still modifying... [id=i-0a27a9f784a88cd40, 00m30s elapsed]
aws_instance.my_server: Modifications complete after 33s [id=i-0a27a9f784a88cd40]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

bucket_name = "260840127026-tc-sample-file-bucket-css-assignment"
current_instance_type = "t2.micro"
instance_id = "i-0a27a9f784a88cd40"

thc   …/css/tf/q10  10:32

thc   …/css/tf/q10  10:32  terraform destroy
```

```
          - type        = "CanonicalUser" -> null
            # (1 unchanged attribute hidden)
        }

      - server_side_encryption_configuration {
          - rule {
              - bucket_key_enabled = false -> null

              - apply_server_side_encryption_by_default {
                  - sse_algorithm     = "AES256" -> null
                    # (1 unchanged attribute hidden)
                }
            }
        }

      - versioning {
          - enabled    = false -> null
          - mfa_delete = false -> null
        }
    }

Plan: 0 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  - bucket_name          = "260840127026-tc-sample-file-bucket-css-assignment" -> null
  - current_instance_type = "t2.micro" -> null
  - instance_id          = "i-0a27a9f784a88cd40" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_s3_bucket.my_bucket: Destroying... [id=260840127026-tc-sample-file-bucket-css-assignment]
aws_instance.my_server: Destroying... [id=i-0a27a9f784a88cd40]
aws_s3_bucket.my_bucket: Destruction complete after 0s
aws_instance.my_server: Still destroying... [id=i-0a27a9f784a88cd40, 00m10s elapsed]
aws_instance.my_server: Still destroying... [id=i-0a27a9f784a88cd40, 00m20s elapsed]
aws_instance.my_server: Still destroying... [id=i-0a27a9f784a88cd40, 00m30s elapsed]
aws_instance.my_server: Destruction complete after 30s

Destroy complete! Resources: 2 destroyed.

thc   …/css/tf/q10  10:32  nvim main.tf

thc   …/css/tf/q10  10:34
```